Documentation technique

NoWave

Architecture et contrats de l'application maritime

Vadim Gavet, Jonathan Cahoreau et Brice Fontaine

Référence technique du 28 octobre 2026
Comparaison et corrections du 30 octobre 2026

Dossier de documentation

Sommaire

Périmètre et principales user stories 4

Autres priorités et guide du document 5

Références du prototype et des wireframes 6

Vue d'ensemble des wireframes 8

Wireframe de validation de position 9

Wireframes d'administration 10

Comportement de l'application en situation dégradée 11

Architecture du système 12

Composants Flutter et interactions 14

Décisions techniques et justification 15

Alternatives et compromis techniques 16

Règles de positionnement des signalements et de cartographie 18

Alertes, identité et confidentialité 21

UML comptes, appareils et signalements 23

UML signalements photo et alertes 24

UML modération et audit 25

Base de données utilisateurs et bateau 25

Base de données appareils et position 27

Base de données signalements 28

Base de données mesures de positionnement photo 29

Base de données stockage des photos 30

Base de données notifications et historique 31

Base de données actions de modération 32

Intégrité de la base de données et valeurs partagées 33

UML services et dépendances 35

Principales méthodes des services 36

UML interfaces externes 37

Séquence 1 publier un signalement photo 38

Séquence 2 consulter la carte 39

Séquence 3 alerter les utilisateurs à proximité 40

Index des routes API 41

Initialisation du profil 43

Préférences du compte et règles partagées 43

Enregistrement de l’appareil et position courante 45

Créer et lire des signalements (routes implémentées) 46

Positionnement par photo et envoi 47

Historiques, accès cartographique et messages push 48

Administration et audit 49

Exemples de requêtes de modération 51

Services externes et intégrations 52

Formats API, confidentialité et erreurs 52

Exigences non fonctionnelles 54

Contrôles qualité 55

Gestion du code source et définition de terminé 56

Annexe des schémas techniques 58

NoWave est une application communautaire de signalement maritime. Les navigateurs publient des observations, consultent des signalements sur une carte mondiale et reçoivent des alertes de proximité. Ce document définit les exigences du MVP, les références d'interface, l'architecture, le modèle de données, les contrats API et les critères de validation pour l'implémentation et la revue du Stage 4.

Référence documentaire : branche dev du dépôt FBCIFY/NoWave, commit ad54ffdb2cf359c36cf22b947494195bddcf87fb (28 octobre 2026). Mise à jour des contrats cartographiques, routes API, interfaces web et mobile et mécanismes de livraison vérifiés dans les fichiers sources. Le code intégré ne prouve ni un déploiement sur le VPS ni une recette Android/iOS.

Périmètre et principales user stories

Les user stories Must Have constituent la base du MVP. Les visiteurs créent un compte ou se connectent. Les navigateurs consultent et publient des signalements. Les administrateurs modèrent les signalements et les comptes.

ID	Must Have (indispensable)	Maquette / wireframe
US-01	Créer un compte pour accéder à NoWave.	US-01 / US-02
US-02	Se connecter avec son e-mail et son mot de passe.	M-02
US-03	Voir sa position GPS actuelle sur la carte.	M-05
US-04	Publier un signalement manuel.	M-04 / M-06
US-05	Voir les signalements sur la carte.	M-05
US-06	Choisir la catégorie d'un signalement.	M-04 / M-06
US-07	Utiliser la caméra pour estimer la position d'un objet.	M-07
US-08	Corriger la position estimée avant publication.	US-08
US-09	Recevoir des alertes pour les signalements situés dans un rayon de 10 NM.	M-09
US-10	Ouvrir directement la carte depuis une notification FCM ; voir le signalement et sa position, avec cadrage automatique et détail sur demande.	M-09 / M-10
US-11	Lire la catégorie, la distance, la date, la photo et l'auteur d'un signalement.	M-11
US-12	Supprimer les faux signalements en tant qu'administrateur.	M-12 US-14
US-13	Réinitialiser un mot de passe oublié.	US-04
US-19	Suspendre les comptes abusifs en tant qu'administrateur.	US-15
US-29	Gérer les signalements et les comptes via une interface d'administration.	M-12 US-14/15

M = maquette numérotée, WF = libellé du wireframe d'origine, - = aucun écran dédié. Les références indiquent les écrans associés, pas l'intégralité des critères d'acceptation.

Les signalements sont des observations communautaires et les positions peuvent être estimées. NoWave ne remplace ni les systèmes officiels de navigation ni les services de sécurité maritime.

Autres priorités et guide du document

Ces priorités reprennent le registre des user stories NoWave. Elles décrivent le périmètre, sans ajouter de date de livraison ni promettre une implémentation ultérieure.

ID	Priorité	Besoin utilisateur	Maquette / wireframe
US-14	Should Have (souhaitable)	Modifier mes propres signalements.	-
US-15	Should Have (souhaitable)	Supprimer mes propres signalements.	-
US-16	Should Have (souhaitable)	Voir l'historique de mes signalements des 30 derniers jours.	US-12
US-17	Should Have (souhaitable)	Distinguer les signalements actifs et expirés.	-
US-18	Should Have (souhaitable)	Regrouper les signalements proches pour garder la carte lisible.	-
US-20	Could Have (optionnel)	Se connecter avec Google, Apple ou Facebook.	-
US-21	Could Have (optionnel)	Personnaliser le profil, l'icône du bateau et la photo de profil.	US-13
US-22	Could Have (optionnel)	Ajouter une licence de navigation au profil.	US-13
US-23	Won't Have (hors MVP)	Reconnaître les animaux ou objets avec l'IA.	-
US-24	Won't Have (hors MVP)	Afficher un score de réputation utilisateur.	-
US-25	Won't Have (hors MVP)	Calculer des itinéraires de navigation.	-
US-26	Won't Have (hors MVP)	Afficher les prévisions et les données météo GRIB.	-
US-28	Won't Have (hors MVP)	Publier des alertes vérifiées provenant des autorités maritimes.	-

M = maquette numérotée, WF = libellé du wireframe d'origine, - = aucun écran dédié. Les références indiquent les écrans associés, pas l'intégralité des critères d'acceptation.

Le journal des signalements personnels (US-16/17, NW-74) est hors du MVP connecté : il ne doit pas être présenté comme disponible. L'historique des alertes de proximité, distinct, est accessible par GET /api/v1/users/me/alerts. Le clustering cartographique lié au zoom ne vaut pas réalisation de l'option de regroupement par catégorie à 200 m (NW-75).

Ordre de lecture

Partie	Contenu
Interface utilisateur	Prototype Figma interactif, wireframes, maquettes mobiles, administration et comportement des permissions.

Partie	Contenu
Architecture	Composants, onze décisions techniques justifiées, règles des signalements, alertes et confidentialité.
Données et UML	Relations entre classes, dictionnaire complet, états et dépendances entre services.
Séquences principales	Publication d'un signalement photo, consultation de la carte et alertes de proximité.
API	Index des routes, initialisation, requêtes, réponses, services externes et erreurs.
Qualité et gestion du code source	Exigences non fonctionnelles, contrôles, branches, revues et définition de terminé.

Références du prototype et des wireframes

Le prototype interactif Figma Make et le PDF NoWave Wireframes complètent les spécifications d'interface. Utilisez le prototype pour explorer les écrans et les transitions, et les wireframes pour comparer la mise en page et la hiérarchie des informations. Les exigences et les contrats API de ce document constituent la référence d'implémentation.

Ouvrir les maquettes interactives NoWave dans Figma

Prototype : Projet MVP Holberton. Wireframes : NoWave Wireframes.pdf, une planche d'ensemble contenant 15 écrans libellés. Revu le 13 septembre 2026

Correspondance des écrans

Les libellés US imprimés sous les wireframes identifient des écrans et ne correspondent pas toujours au registre des user stories. Le tableau ci-dessous relie leur contenu visible aux user stories de référence utilisées dans ce rapport.

Libellé du wireframe	Écran visible	User story de référence
US-01 à US-04	Accueil, inscription, connexion, réinitialisation du mot de passe	US-01, US-02, US-13
US-05 et US-06	Tableau de bord et carte	US-03, US-05
US-07 et US-09	Vues caméra, US-09 répète la capture	US-07, le formulaire de signalement se rapporte à US-04/06
US-08	Confirmer ou corriger la position estimée	US-08
US-10	Liste des alertes	US-09
US-11	Détails du signalement	US-11
US-12	Journal de bord personnel et analyse	US-16, US-17 pour l'historique
US-13	Informations de profil et de bateau	US-21, US-22 pour les fonctions optionnelles
US-14 et US-15	Détails d'un signalement admin et d'un compte suspendu	US-12, US-19, US-29

L'interaction notification-vers-carte de l'US-10 est illustrée par le prototype et la maquette M-10 existante. Un écran caméra seul ne démontre pas un formulaire de publication complet.

Règles d'interface pour l'implémentation

L'inscription demande un nom d'utilisateur unique obligatoire pour POST /users/me après la création du compte Firebase. Le journal de bord personnel limité à 30 jours appartient aux maquettes et reste hors du MVP connecté.

Les cinq actions de modération exigent un motif. La visibilité de l'auteur et du bateau reste optionnelle. Les distances et les dates des maquettes sont des exemples et ne prouvent ni l'éligibilité d'une alerte ni le fait qu'un signalement n'est pas expiré.

Le texte d'accueil sur les animaux marins représente une catégorie de signalement ; le MVP couvre également la pollution et les obstacles. Les widgets météo, les champs de licence et les exemples d'analyses n'élargissent pas les priorités définies dans le registre des user stories. La cartographie intégrée à dev utilise le SDK Mapbox avec le style Mapbox Standard importé localement, enrichi par les couches marines NoWave et une couche MVT de signalements distincte, et les coordonnées respectent les conventions d'affichage/API documentées.

Depuis un tap explicite sur une notification FCM (action « Ouvrir » lorsque l'application est active, tap en arrière-plan ou à son lancement), NoWave ouvre directement la carte principale existante, sans afficher automatiquement de fiche de signalement ni créer une seconde instance de carte. Le mobile identifie le report_id dans le push, puis relit le signalement courant par GET /api/v1/reports/{report_id} avant de placer son marqueur ; ni la position ni les autres données du payload FCM ne remplacent les données vérifiées du backend.

La carte met en évidence le marqueur du signalement sélectionné, indépendamment du chargement éventuel de sa tuile MVT, et la position actuelle de l'utilisateur lorsque la localisation du téléphone est exploitable. La caméra est cadrée automatiquement sur les deux positions avec une marge adaptée à l'écran et aux contrôles (limites géographiques, sans zoom fixe) ; à courte distance elle rapproche les repères sans les confondre, et à plus grande distance elle dézoome pour garder les deux visibles. La position utilisateur est représentée par une flèche orientée uniquement lorsqu'un cap fiable est disponible, sinon par un repère neutre. Le suivi de position ne doit pas annuler le cadrage en recentrant aussitôt sur l'utilisateur ; après l'ajustement initial, déplacement, zoom et rotation manuels restent libres. La fiche détaillée s'ouvre uniquement si l'utilisateur touche volontairement le marqueur.

Deux notifications FCM correspondant à des événements distincts mettent successivement à jour le marqueur sélectionné et le cadrage de la même carte, sans empiler les fiches ; les doublons du même événement restent dédupliqués. Si une tentative de navigation ou de cadrage échoue, une nouvelle tentative explicite doit être possible sans recevoir une seconde notification, sans boucle de navigation ni perte des protections contre les doublons. Le bouton Retour suit la navigation normale de l'application.

Cas de repli : si la localisation du téléphone est refusée, désactivée ou indisponible, centrer la carte sur le seul signalement avec un zoom lisible et indiquer « Position indisponible — cadrage sur le signalement ». Si une position exploitable arrive ensuite pendant ce parcours, effectuer au plus un recadrage adapté, sans recentrages répétés. Un cap absent ne doit jamais être simulé. Si le signalement n'est plus accessible (404, expiration, retrait, refus) ou si le réseau échoue, ne pas cibler de coordonnées inventées : afficher un message explicite sur le parcours carte et proposer « Réessayer » ou l'historique. Le MVP ne garantit pas la consultation hors ligne.

Préservation des parcours : si un signalement est en cours de création ou de positionnement, ne pas déplacer son point ni fermer son formulaire sans l'accord de l'utilisateur ; différer le cadrage ou demander de terminer/annuler le formulaire. Les alertes choisies depuis l'historique peuvent toujours ouvrir la fiche détaillée existante ; son action « Voir sur la carte » réutilise la carte principale. Le changement de compte, la déconnexion, la suppression ou la suspension empêchent d'ouvrir une notification appartenant à une ancienne session. Le texte de la notification (NW-162), les contrats FCM/API, le worker et la base de données restent inchangés par NW-163.

Vue d’ensemble des wireframes

NoWave Wireframes.pdf présente le parcours mobile et deux panneaux d’administration. Les libellés des écrans ci-dessous reprennent la source ; utilisez le tableau de correspondance précédent pour assurer la traçabilité avec les user stories.

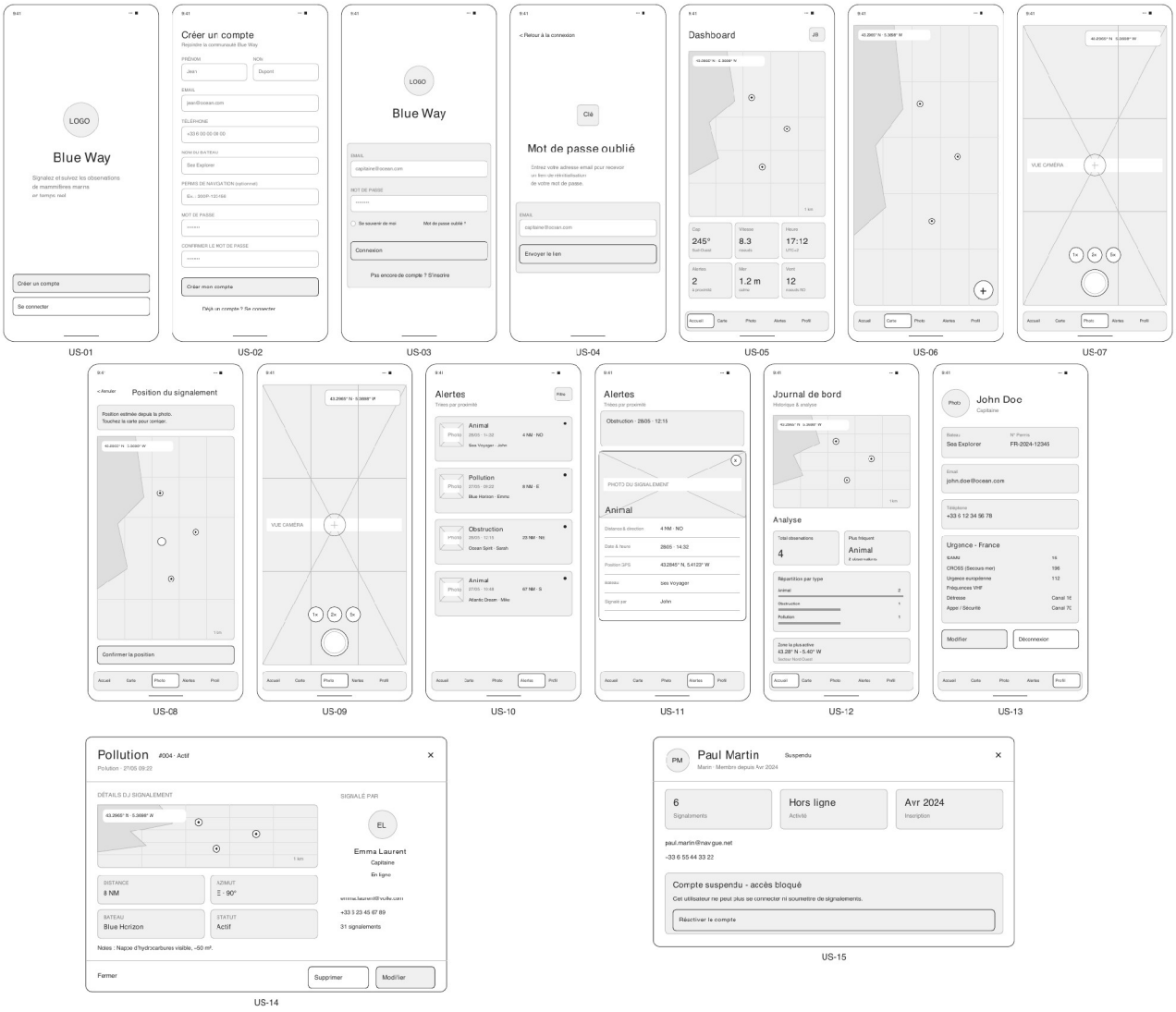
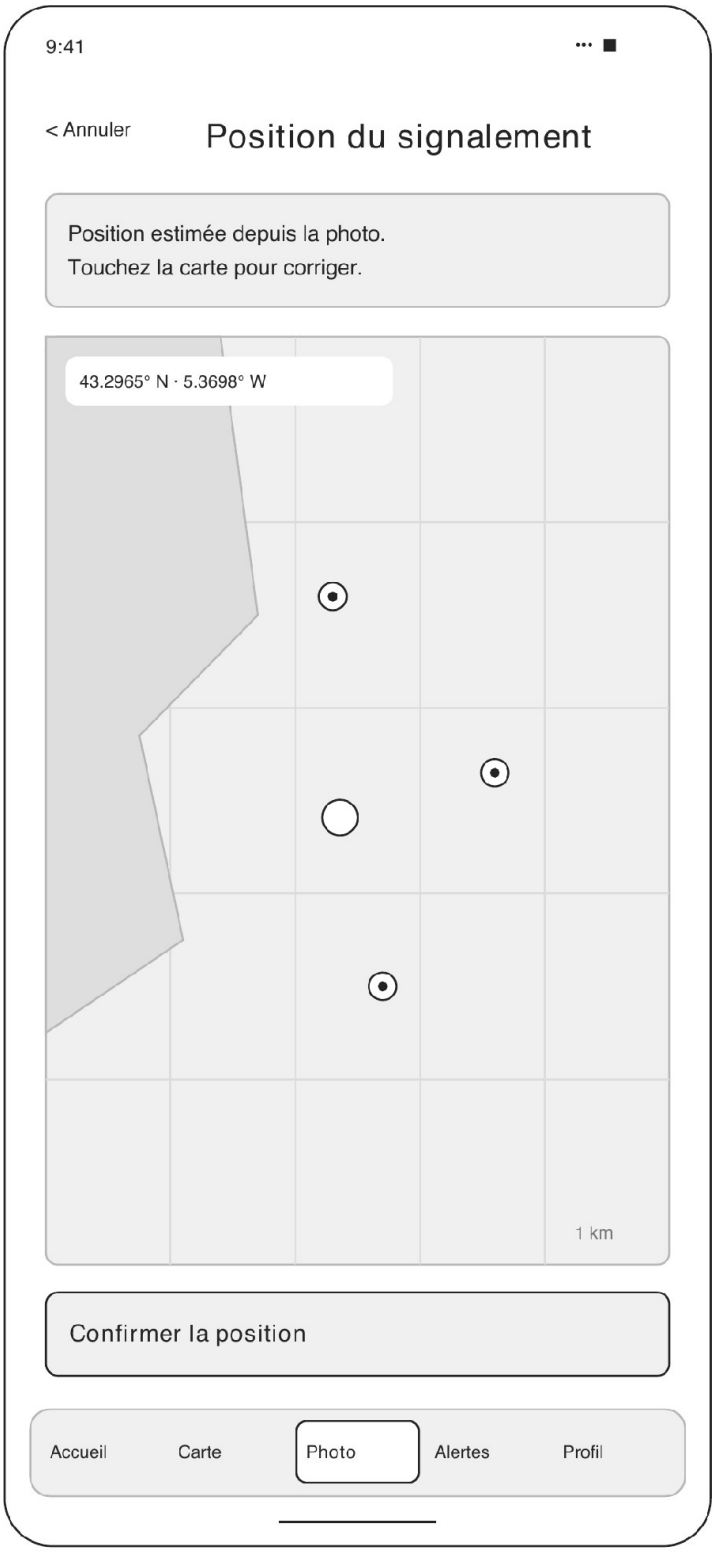


Planche complète des wireframes. Les agrandissements des pages suivantes mettent en évidence la validation de position et la modération.

Wireframe de validation de position

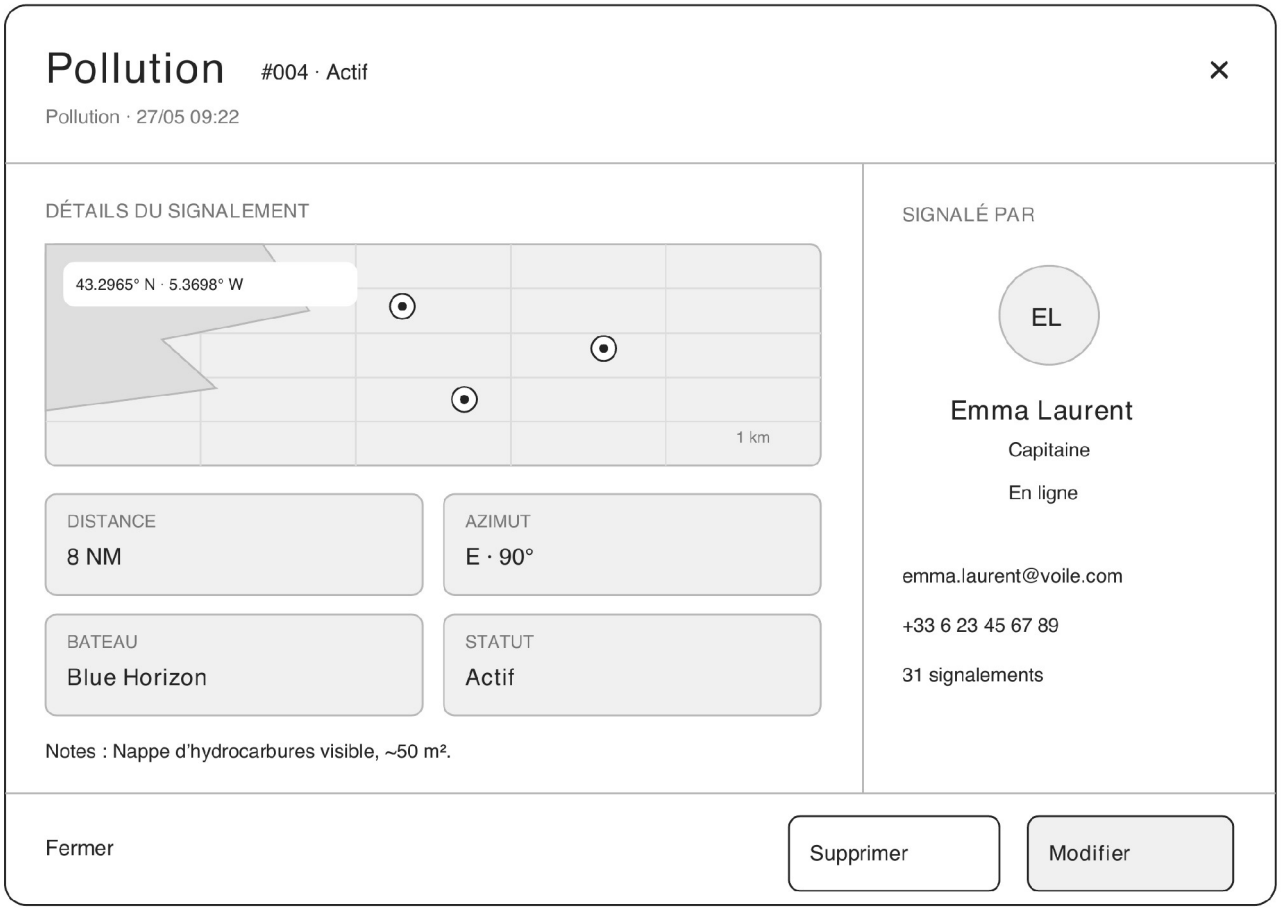


US-08

Correction de position, wireframe US-08. L'utilisateur confirme ou déplace le point estimé avant publication ; final_position enregistre le point confirmé.

Les mesures de capture alimentent PositioningService. Même lorsqu'aucune estimation n'est disponible, l'utilisateur peut définir le point final et poursuivre en mode photo. La publication conserve l'heure d'observation d'origine et enregistre le signalement indépendamment de l'envoi du JPEG.

Wireframes d’administration



US-14

Détail d’un signalement, libellé de wireframe US-14, correspondant aux US-12 et US-29 de référence.



US-15

Compte suspendu, libellé de wireframe US-15, correspondant aux US-19 et US-29 de référence. Les cinq actions de modération exigent un motif comme indiqué dans la section Administration et audit.

Comportement de l’application en situation dégradée

Les règles suivantes définissent la réaction de l’application mobile lorsqu’une capacité de l’appareil, le réseau ou un service dépendant est indisponible. Elles complètent les wireframes en précisant les comportements de repli visibles par l’utilisateur.

Situation	Comportement
GPS indisponible	Expliquer les limites et désactiver les fonctions nécessitant le GPS. Pour l’ouverture d’une alerte FCM, afficher le signalement seul avec un message de position indisponible, puis recadrer une fois si la position revient. Conserver visibles les données en cache disponibles sans promettre un fonctionnement hors ligne.
Précision GPS supérieure à 50 m	Maintenir la capture photo désactivée et continuer à rechercher une mesure exploitable.
Caméra indisponible	Afficher un message clair. Le signalement manuel reste disponible.
Réseau indisponible	Informer l'utilisateur que la publication et le rafraîchissement de la carte nécessitent le réseau. Lorsqu'un signalement photo a déjà été publié mais que l'envoi du JPEG échoue, le fichier est conservé temporairement uniquement pendant le parcours de publication en cours. L'utilisateur peut réessayer l'envoi avec le même report_id et la même ligne report_photos tant que ce parcours reste ouvert. Si le parcours est fermé avant la réussite de l'envoi, le signalement reste publié mais la photo devient indisponible. Aucune reprise automatique ni synchronisation différée de la photo n'est prévue dans le MVP.
Échec de l’envoi de photo	Conserver le signalement actif. Conserver temporairement le JPEG uniquement pendant le parcours de publication en cours et permettre à l'utilisateur de réessayer l'envoi séparément avec le même report_id et la même ligne report_photos. Si le parcours est fermé avant la réussite de l'envoi, le signalement reste publié mais la photo est indisponible. Aucune reprise automatique après fermeture n'est prévue.

Situation	Comportement
Coordonnées	Afficher les degrés, minutes et secondes. Les coordonnées API utilisent GeoJSON, longitude en premier.
Consentement	Demander séparément l'autorisation pour les alertes de proximité. Le nom d'utilisateur et les informations du bateau sont masqués par défaut.

Architecture du système

NoWave utilise un monolithe modulaire avec une architecture hexagonale. Flutter communique avec un backend FastAPI modulaire unique. Les modules métier partagent un même déploiement ; les ports et adaptateurs isolent la logique de domaine de Firebase, Scaleway, PostgreSQL/PostGIS et des fournisseurs cartographiques. Le déploiement comprend aussi un worker serveur photo_cleanup distinct de NotificationWorker pour entretenir le stockage objet des photos. Le rendu cartographique importe Mapbox Standard ; les tuiles de signalements proviennent de PostGIS, tandis que les données marines publiques sont agrégées par les adaptateurs du backend. Le site web (web/) est construit avec Vite et servi par Nginx. Sa carte publique et l'administration partagent le composant map-view.js ; les pages et actions administratives restent protégées. Les exemples de publication et d'alertes du parcours de présentation sont synthétiques et ne publient aucun signalement ni notification réelle.

Vue d architecture

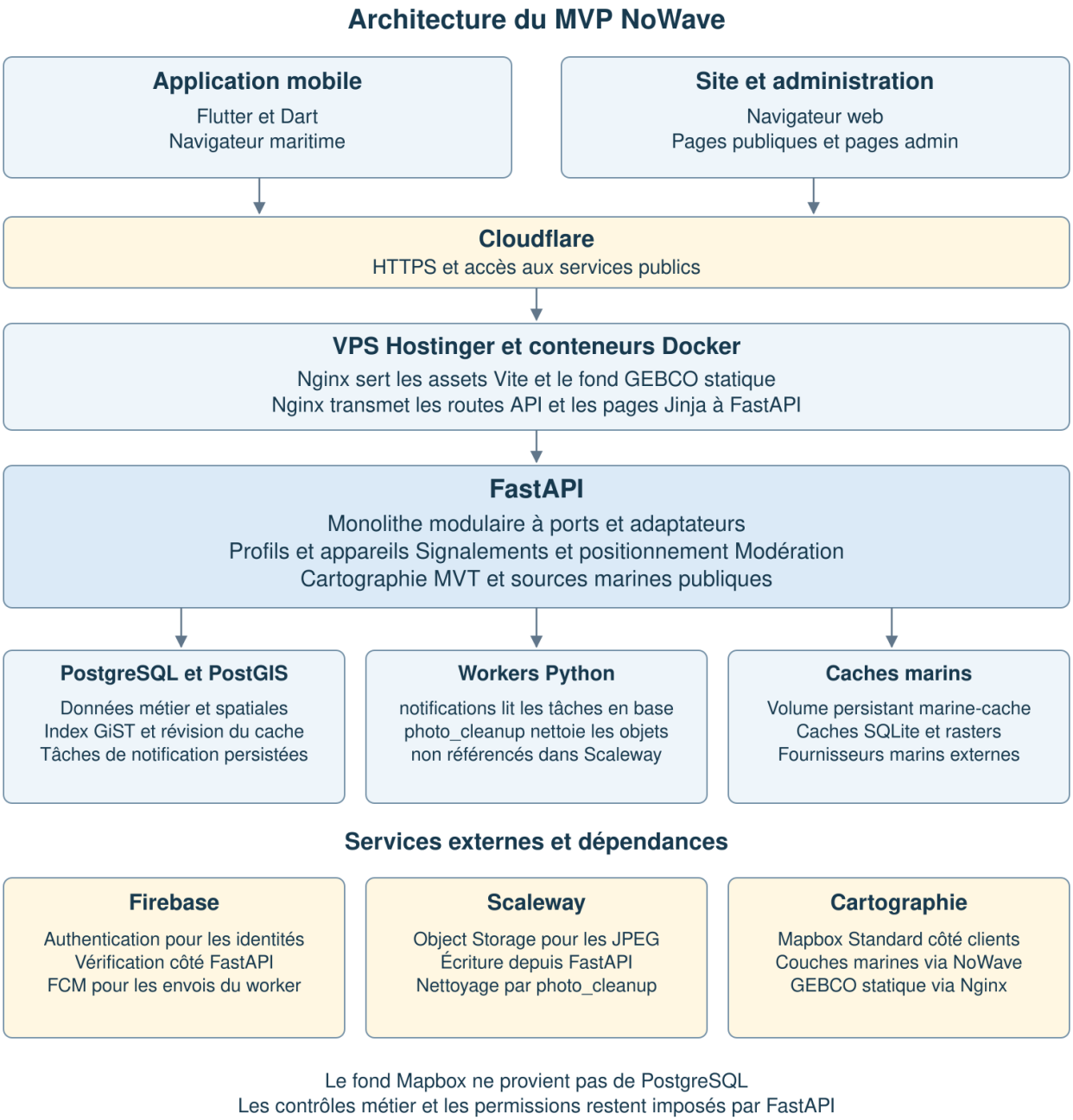


Figure 1. Architecture générale du MVP alignée sur dev. La section Architecture cartographique et données marines détaille les sources, les caches et les règles de chargement.

MCD

Le modèle conceptuel de données présente les entités métier et les associations du MVP. Les cardinalités expriment les règles de relation. Les détails SQL restent définis dans le dictionnaire de données.

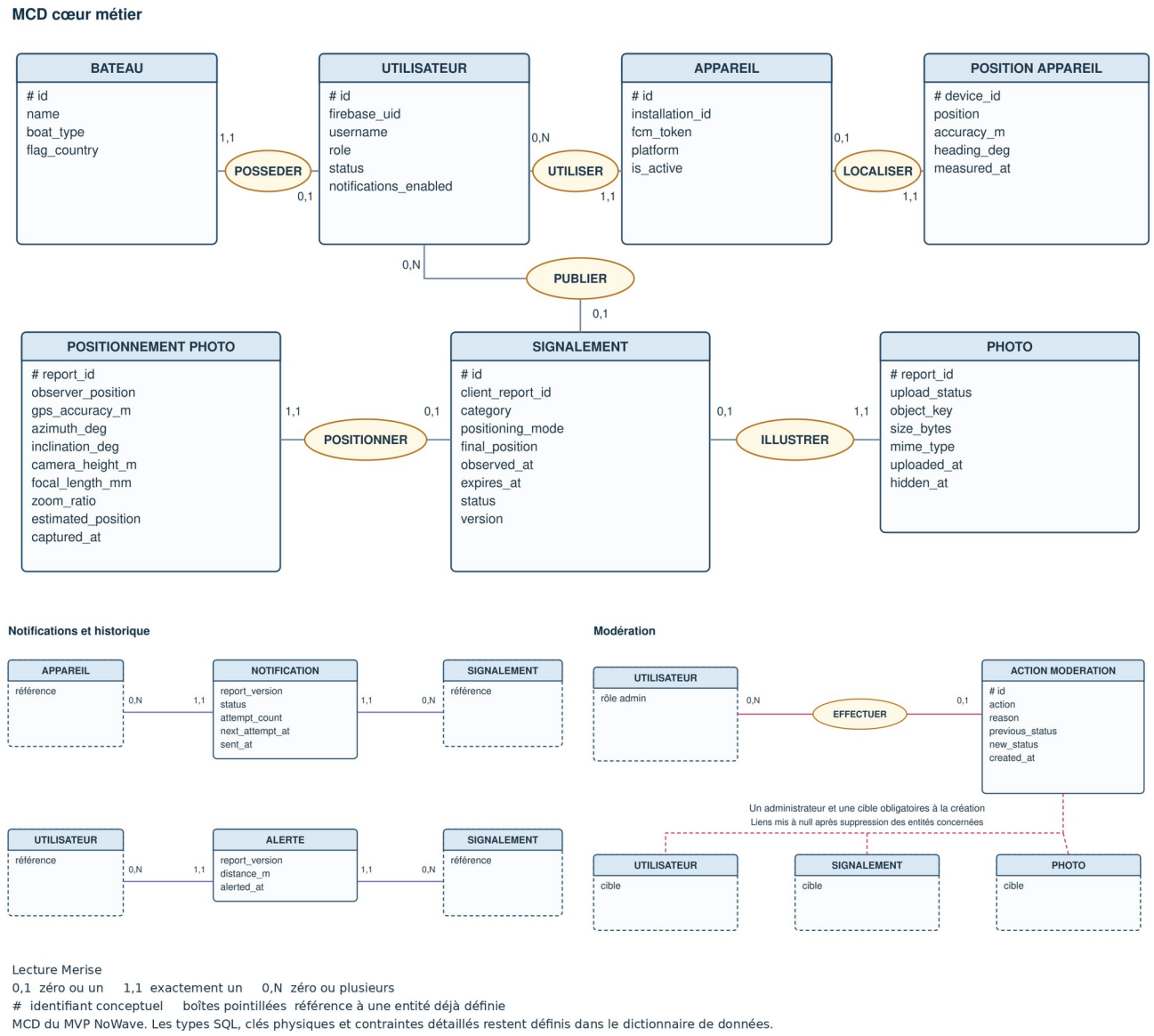


Figure 2. MCD Merise du MVP NoWave

Périmètre du MCD : le modèle relationnel représenté traite les utilisateurs, bateaux, appareils, signalements, photos, alertes et modération. Les sources marines, caches de rendu et snapshots d'affichage ne sont pas des entités persistantes métier dans PostgreSQL ; ils relèvent de l'intégration cartographique décrite ci-après. Les migrations versionnées visibles dans dev vont jusqu'à 20261012_0003_map_cache_revision.py.

Composants Flutter et interactions

Le frontend Flutter est organisé autour des principaux parcours utilisateur. Les écrans délèguent la navigation, l'accès aux fonctions de l'appareil, la persistance locale et les appels API aux contrôleurs et adaptateurs ; les règles métier restent imposées par FastAPI. Le profil, le bateau, les informations personnelles et la suppression du compte s'ouvrent en panneaux au-dessus de la carte. PersonalInformationScreen utilise PATCH /users/me pour modifier ou effacer date_of_birth et nationality ; BoatProfileLoader partage le chargement du bateau pendant une ouverture du profil.

Composant Flutter	Responsabilité	Interactions principales
AppShell, AuthFlow et ProfileScreen	Démarrer l'application, sélectionner les routes authentifiées, terminer l'initialisation et gérer le profil, le bateau et les paramètres de consentement.	Firebase Authentication, POST/GET/PATCH /users/me, /users/me/boat, DeviceController.
MapScreen et MapController	Afficher la position courante, le fond de carte maritime et les signalements NoWave regroupés ; sélectionner et mettre en évidence un signalement FCM, cadrer signalement et position utilisateur quand elle est disponible, et ouvrir le détail uniquement sur action volontaire.	Mapbox Maps Flutter ; style local avec import Mapbox Standard ; couches publiques NoWave (bathymétrie PNG, marques, sondes, isobathes, dangers, ports) et tuiles MVT authentifiées des signalements ; GET /reports/{report_id} et GPS Flutter.
ReportCreationFlow	Collecter les données du signalement manuel ou photo, la catégorie et la position finale, puis publier le signalement.	Adaptateurs caméra/GPS, PositioningController, POST /reports, PhotoUploadService.
PositionCorrectionScreen et PositioningController	Demander un point estimé, l'afficher et permettre à l'utilisateur de confirmer ou déplacer le point final avant publication.	POST /position-estimates, MapController, ReportCreationFlow.
NotificationHandler, DeviceController et PhotoUploadService	Enregistrer l'installation, acheminer les taps FCM vers la carte principale sans fiche automatique, conserver déduplication et nouvelle tentative explicite après échec, et gérer l'envoi des photos. Une nouvelle tentative d'envoi du JPEG est possible tant que le parcours de publication reste ouvert ; aucun brouillon JPEG persistant n'est conservé après fermeture.	PUT /devices/current, PUT /devices/current/position, FCM, MapScreen / MapController, GET /reports/{report_id}, PhotoUploadService.

Décisions techniques et justification

Les décisions suivantes ont été retenues selon les exigences fonctionnelles, les contraintes du MVP, l'environnement d'exploitation attendu et les besoins de maintenabilité de NoWave.

Décision technique	Exigence NoWave	Justification
Flutter / Dart	Android et iOS, GPS, orientation, inclinaison, caméra et stockage local.	Une base de code mobile partagée réduit l'effort d'implémentation et de maintenance tout en conservant l'accès au GPS, à l'orientation, à l'inclinaison, à la caméra et aux fonctions nécessaires au parcours de signalement.
FastAPI / Python	Validation REST, calculs de positionnement, accès à la base de données et services externes.	Les modèles typés de requête/réponse rendent les contrats explicites. Python regroupe les calculs et la logique métier backend dans un même écosystème.
Monolithe modulaire	Petite équipe MVP avec utilisateurs, signalements, positionnement, cartographie, notifications et modération.	Un backend unique est plus simple à déployer, tester et exploiter. Les modules métier internes restent séparés sans le coût de services distribués.
Architecture hexagonale	Dépendances à Firebase, Scaleway, PostgreSQL/PostGIS et aux fournisseurs cartographiques.	Les ports et adaptateurs isolent les règles métier, facilitent les doublures de test et permettent de changer de fournisseur sans réécrire la logique de domaine.

Décision technique	Exigence NoWave	Justification
PostgreSQL / PostGIS	Enregistrements liés et requêtes géographiques pour les alertes à 10 NM, le regroupement à 200 m et la consultation de la carte.	Les transactions, clés étrangères et contraintes préservent l'intégrité. Les types géographiques, fonctions spatiales et index GiST prennent en charge les requêtes de distance requises.
Mapbox Standard + couches marines NoWave	Carte maritime interactive avec des données de signalements détenues indépendamment.	Mapbox assure le rendu et fournit le fond Standard. FastAPI assemble les ressources marines publiques provenant de fournisseurs externes ; les signalements sont fournis séparément en MVT depuis PostGIS. Les flux marins et métier ont des contrats, caches et règles d'accès distincts.
Couche NoWave MVT	Consultation mondiale sans télécharger tous les signalements actifs.	FastAPI/PostGIS génère des tuiles pour les zones z/x/y visibles. Les données compactes et le regroupement à l'affichage réduisent les transferts tout en conservant les signalements individuels.
Scaleway Object Storage	Stockage des JPEG sans agrandir la base relationnelle.	Le stockage objet conserve les octets des images ; PostgreSQL conserve les métadonnées et les clés d'objet. L'envoi indépendant permet à la publication de survivre à une panne temporaire du stockage. Le bucket est privé, les URL de lecture sont signées et temporaires, et un worker serveur supprime les objets orphelins selon la politique de purge.
Firebase Authentication + FCM	Identité et notifications push sur Android et iOS.	Firebase gère l'identité ; FastAPI contrôle les rôles, le statut et l'autorisation. FCM transporte les messages après que FastAPI/PostGIS a sélectionné les destinataires éligibles.
Hostinger VPS + Docker + Cloudflare	Backend FastAPI et PostgreSQL/PostGIS déployés dans des conteneurs Docker sur un VPS. Cloudflare protège les services publics exposés par NoWave.	Cette architecture donne à l'équipe le contrôle du déploiement. PostgreSQL reste privé et n'est pas exposé directement sur Internet. Les secrets sont configurés hors du dépôt Git.
Interface d'administration HTML / Jinja	Modération des signalements et gestion des comptes pour les administrateurs.	FastAPI sert une petite interface sans seconde pile frontend complète, ce qui réduit le travail de déploiement et de maintenance pour le MVP.

Alternatives et compromis techniques

Nous conservons la pile technique sélectionnée car elle correspond au périmètre du MVP, aux règles relationnelles et spatiales, ainsi qu'à la capacité d'une équipe de trois personnes. Les alternatives ci-dessous restent crédibles. La comparaison explique leur adéquation architecturale et les limites acceptées ; elle ne constitue ni un benchmark ni un classement par coût minimal.

Choix retenu	Alternative et avantage	Pourquoi ce choix est retenu et compromis accepté
Flutter / Dart	React Native propose aussi une application Android/iOS partagée, basée sur React et JavaScript. Des applications natives séparées offrent un contrôle direct de chaque plateforme mais exigent deux implémentations UI.	Nous retenons Flutter pour une UI mobile cohérente et des flux partagés de signalement, caméra, géolocalisation et cartographie. React Native pourrait également répondre à ces besoins. Limite acceptée : dépendances Dart et plugins, avec intégration native et tests encore nécessaires pour la caméra, le GPS et les capteurs. À reconsidérer si une capacité indispensable de l'appareil n'est pas suffisamment prise en charge.

Choix retenu	Alternative et avantage	Pourquoi ce choix est retenu et compromis accepté
FastAPI / Python	Django REST Framework fournit des serializers et des viewsets dans Django. C'est une alternative Python crédible lorsqu'un framework applicatif plus large est souhaité.	Nous retenons FastAPI pour des contrats API typés explicites et des calculs de positionnement dans le même backend Python. Cela convient à une API principalement mobile avec une petite interface de modération. Limite acceptée : l'intégration base de données, l'autorisation et les workflows d'administration doivent encore être assemblés et testés avec soin. Django devient plus intéressant si un back-office important devient la charge principale.
Monolithe modulaire	Les microservices permettent un déploiement et une mise à l'échelle indépendants des capacités métier. Ils introduisent aussi des frontières réseau et des problématiques opérationnelles distribuées.	Nous retenons un backend modulaire unique pour une équipe MVP de trois personnes. La création du signalement, les lignes associées et les tâches d'alerte peuvent être validées dans une seule transaction de base de données. Limite acceptée : les modules partagent un déploiement et ne peuvent pas tous évoluer indépendamment. N'extraire un service que lorsque la charge mesurée, l'isolation ou une responsabilité séparée justifie la coordination supplémentaire.
Architecture hexagonale	Une application en couches classique avec des appels SDK directs nécessite moins d'interfaces au départ. Elle peut suffire pour une petite application ayant peu de dépendances externes.	Nous retenons les ports et adaptateurs car l'identité, le stockage photo et le push sont des dépendances externes, tandis que le positionnement et l'éligibilité aux alertes doivent rester des règles métier testables. Limite acceptée : davantage d'interfaces et de câblage. Garder les ports aux frontières significatives et éviter de créer une abstraction pour chaque helper. Cette organisation complète le monolithe et n'impose pas de microservices.
PostgreSQL / PostGIS	MongoDB prend en charge les index géospatiaux et les requêtes de proximité ; les données géographiques seules ne l'excluent donc pas. Son modèle documentaire constitue une autre approche de persistance.	Nous retenons PostgreSQL/PostGIS pour le modèle relationnel existant de 10 tables, les opérations atomiques de signalement et d'audit, les distances indexées en mètres et la génération MVT. Il réunit ces besoins dans une seule base. Limite acceptée : migrations de schéma, réglage des index spatiaux et opérations de base de données. Revoir le modèle si des workloads orientés documents deviennent plus importants que ces relations.
Mapbox Standard + couches marines NoWave	MapLibre Native est un moteur de rendu vectoriel alternatif ; il peut être évalué avec un binding Flutter adapté et le même fournisseur de données maritimes. Le moteur de rendu et le fond de carte sont deux choix distincts.	Le code retient Mapbox Maps Flutter et un style local important Mapbox Standard. Les données nautiques sont ajoutées sous forme de couches NoWave issues de fournisseurs marins, sans réutiliser MapTiler Ocean. Limites à valider : attribution des sources, zones couvertes, coûts et rendu Android/iOS ; MapLibre reste une alternative non implémentée.
Couche NoWave MVT	Un endpoint GeoJSON basé sur une bounding box est plus simple à inspecter et peut suffire avec un faible nombre de signalements. Les deux approches peuvent limiter les données à la zone visible de la carte.	Nous retenons des MVT générées côté serveur pour la consultation mondiale avec chargement par tuiles et regroupement à l'affichage. Les détails du signalement restent dans l'API JSON. Limite acceptée : génération des tuiles et invalidation du cache plus complexes. Le rafraîchissement en ligne doit respecter les suppressions et l'expiration à 24 heures. Comparer les tailles de payload et le temps de rendu avant d'affirmer un gain de vitesse mesuré.

Choix retenu	Alternative et avantage	Pourquoi ce choix est retenu et compromis accepté
Scaleway Object Storage	Amazon S3 est une alternative crédible de stockage objet managé. Conserver les octets JPEG dans la base éviterait un service de stockage séparé mais augmenterait la taille de la base et des sauvegardes.	Nous retenons Scaleway comme stockage photo S3-compatible derrière PhotoStoragePort, séparé des métadonnées de signalement. Cela prend en charge le modèle d'envoi indépendant et de retry. Limite acceptée : un fournisseur supplémentaire, des identifiants et une gestion des pannes. S3 pourrait satisfaire la même exigence de base ; aucun avantage de prix ou de performance n'est démontré ici. Réévaluer selon la région, le coût de transfert et les exigences de disponibilité.
Firebase Authentication + FCM	Supabase Auth propose un autre service d'identité managé. OneSignal propose l'orchestration et l'analyse des push au-dessus des services de livraison de plateforme, dont FCM et APNs. Ce sont des alternatives séparées pour l'identité et la messagerie.	Nous retenons les SDK Firebase pour l'identité et le transport FCM, tandis que FastAPI/PostGIS gère les permissions et la sélection des destinataires. L'intégration directe à FCM évite un broker de push supplémentaire pour le flux actuel d'alertes de proximité. Limite acceptée : dépendance au fournisseur et contraintes de livraison mobile ; l'acceptation d'une requête d'envoi ne prouve pas sa réception. Reconsidérer si les besoins d'identité ou d'orchestration des notifications dépassent ce périmètre.
Hostinger VPS + Docker + Cloudflare	Une plateforme PaaS comme Railway ou Render réduit l'administration de l'hôte. Un VPS autogéré donne davantage de contrôle mais rend l'équipe responsable de son exploitation et de sa sécurisation.	Nous retenons un VPS Hostinger avec Docker pour déployer FastAPI et PostgreSQL/PostGIS dans des conteneurs séparés. Cloudflare protège les services publics. PostgreSQL n'est pas exposé directement sur Internet. Les secrets restent hors du dépôt. Les sauvegardes, la restauration, les migrations, les contrôles de santé et le rollback doivent être documentés et testés.
Interface d'administration HTML / Jinja	Une application monopage React offre des composants interactifs réutilisables et un état côté client. C'est une alternative crédible pour un espace de modération plus riche.	Nous retenons des pages Jinja rendues côté serveur pour les listes de signalements, les vues de détail et les formulaires de modération actuels, en utilisant le backend FastAPI existant. Cela évite une application SPA séparée et une couche supplémentaire de gestion d'état. Limite acceptée : les interactions temps réel complexes nécessitent davantage de JavaScript. Reconsidérer React si l'interface d'administration exige une collaboration temps réel importante ou des workflows côté client plus riches.

Règles de positionnement des signalements et de cartographie

Sujet	Spécification
Carte mondiale	Flutter importe Mapbox Standard dans le style local en projection globe. Les couches publiques NoWave affichent la bathymétrie, l'hydrographie et les marques nautiques selon le zoom. La couche des signalements MVT est séparée, authentifiée et lue depuis PostGIS. Le détail complet provient de GET /reports/{report_id}.
Position de l'appareil	Envoyer la première position exploitable immédiatement, puis au plus une fois toutes les 60 secondes. Après un envoi accepté, ne pas renvoyer une mesure située à moins de 50 m, sauf si sa précision est au moins deux fois meilleure. Lorsque les alertes de proximité sont activées, le suivi continue en arrière-plan tant que l'application n'est pas fermée de force. Le filtre d'envoi ne modifie pas le flux GPS local utilisé par la carte et la caméra. Ne stocker que la dernière mesure exploitable. La précision doit être finie et comprise entre 0 et 50 m. Le cap fait pivoter l'icône.

Sujet	Spécification
Signalement manuel	L'utilisateur choisit final_position et une catégorie. Aucune mesure de positionnement ni ligne photo n'est créée.
Signalement photo	Conserver localement un JPEG compressé, collecter les mesures, demander une estimation, puis laisser l'utilisateur confirmer ou corriger final_position. Un échec d'estimation conserve le signalement en mode photo.
Heure d'observation	Conserver le observed_at réel de l'observation. Rejeter les observations futures et celles datant de 24 heures ou plus. expires_at = observed_at + 24 heures.
Visibilité sur la carte	Seuls les signalements actifs dont expires_at est postérieur à l'heure courante apparaissent sur la carte active.
Idempotence	Réutiliser client_report_id après une interruption réseau afin d'éviter de créer deux fois le même signalement et la même vague d'alertes.
Photos	JPEG uniquement, maximum 500 000 octets. Flutter supprime les EXIF et FastAPI vérifie le fichier réel, sa taille et son contenu image avant envoi.
Regroupement	Les observations de même catégorie situées à moins de 200 m peuvent être regroupées visuellement. Le clustering MVT lié au niveau de zoom est distinct. Les deux mécanismes conservent les enregistrements individuels en base.

Architecture cartographique et données marines. état de dev (28/10/2026)

Fond de carte : Flutter charge un style local GlobeStyle qui importe mapbox://styles/mapbox/standard et utilise la projection globe. MAPTILER_STYLE_URL n'est plus utilisé par ce parcours. Le fond Mapbox n'est pas généré par PostgreSQL. Les couches marines NoWave se superposent au fond ; les signalements utilisateur restent une source vectorielle MVT authentifiée et distincte.

Chaîne des données marines : les routes FastAPI /api/v1/map/{famille}/… sollicitent les services marine_tiles et marine_merge, puis les adaptateurs fournisseurs. Les géométries WGS-84 sont normalisées, fusionnées et encodées en MVT, GeoJSON ou PNG de bathymétrie selon la famille. Les données hydrographiques sont destinées à l'affichage et ne remplacent pas les documents nautiques officiels. Les fournisseurs ne couvrent pas nécessairement toutes les zones.

Fond mondial : la pyramide GEBCO 2026 pré-rendue Z0–Z6 (gebco-2026-v2, WebP) est livrée dans l'image web et servie directement par Nginx sous /api/v1/map/seafloor/gebco-2026-v2/{z}/{x}/{y}.webp, avec un cache d'un an. Elle ne sollicite ni FastAPI ni les fournisseurs pendant la consultation. Son relief grossier reste visuel et ne produit ni sondes ni isobathes. Le détail régional EMODnet se superpose dès Z5 ; son emprise est limitée à [−73,125 ; 5,625 ; 45 ; 85,051129] (ouest, sud, est, nord). Les deux rasters préchargent des tuiles parentes avec prefetchZoomDelta=4 ; le détail régional applique un délai réseau de 600 ms.

L'application sépare API_BASE_URL (profil, signalements, photos, notifications, MVT authentifiées) de MARINE_API_BASE_URL (facultative, uniquement les sources marines publiques). À défaut de MARINE_API_BASE_URL, les routes marines utilisent API_BASE_URL. Les requêtes marines publiques n'envoient pas de jeton Firebase vers un autre hôte. La configuration est injectée à la compilation via --dart-define-from-file et impose une nouvelle compilation lorsqu'elle change.

Règles de zoom et de visibilité implémentées

Famille	Zoom / chargement	Comportement
Bathymétrie raster	Dès Z5	Détail régional PNG sur le fond mondial GEBCO ; masques côtiers et gestion des données absentes.
Ports / mouillages	Préchargement Z6 ; affichage Z8	Enrichissement des données portuaires à partir de Z10.

Famille	Zoom / chargement	Comportement
Marques maritimes et balisage	Préchargement Z10 ; 0 % à Z10 et 100 % à Z12	Opacité interpolée ; catégories de marques distinguées par leurs propriétés et icônes.
Sondes et isobathes	Préchargement Z12 ; affichage Z13	Les contours d'affichage sont maintenus sur une grille de snapshots Z12 pour préserver leurs identifiants et géométries.

Persistance visuelle : MarineViewport conserve des snapshots de cellules et réutilise les encodages GeoJSON lorsqu'ils sont rigoureusement inchangés ; chaque réponse complète remplace sa zone dans MarineMosaic. Les anciennes géométries couvrent les trous tant que la relève utile n'est pas complète. Une réponse complète vide confirme une absence ; une réponse partielle, même vide, ou une erreur ne retire pas l'ancienne couverture. Les cellules remplacées ou sorties de la région utile sont libérées. Le contrôleur de rotation du globe déplace la longitude à 2 degrés/s, avec une cadence minimale de mise à jour de 33 ms lorsque les conditions de visibilité, d'interaction et de réduction des animations le permettent (plage de zoom admissible du contrôleur : Z0 à Z4,5). Les effets d'atmosphère et d'étoiles relèvent de la présentation Flutter/Mapbox.

Chargement progressif : la demande suit les limites réelles de la caméra et une marge d'une cellule, y compris à l'antiméridien. Le plan vise d'abord 48 cellules visibles, puis utilise les places restantes pour le détail central et la marge, avec un maximum de 64 cellules et deux téléchargements GeoJSON simultanés. Le cache est borné à 64 snapshots / 12 Mio encodés, plus 8 000 points conservés / 4 Mio encodés ; ces budgets excluent les objets Dart et les buffers GPU. La résolution de base est adaptée au nombre de cellules, sans modifier la grille Z12 des contours. L'admission libère atomiquement les anciennes cellules inutiles et les marges moins prioritaires ; si le remplacement ne tient pas en mémoire, la cellule est différée sans abaisser toute sa famille ni retélécharger à chaque temporisation. Les requêtes dépassées sont annulées, le travail est suspendu en arrière-plan et les erreurs ou réponses partielles sont réessayées après environ 15 secondes.

Caches, dégradation et limites de réseau

Signalements : GET /api/v1/map/tiles/{z}/{x}/{y}.mvt renvoie Cache-Control: private, max-age=15, must-revalidate et Vary: Authorization. Tuiles marines vectorielles publiques : max-age=300 s en résultat normal, 15 s si réponse dégradée ; la famille soundings est toujours no-store. Bathymétrie PNG : max-age=86400 s si résultat complet ; une image dont le contexte est incomplet ne doit pas être conservée (no-store). Une indisponibilité marine renvoie HTTP 503 en JSON, Cache-Control: no-store, X-NoWave-Marine-Status: unavailable et Retry-After: 15, pour les routes GeoJSON, MVT et PNG. Une panne ne devient pas une tuile transparente réussie. Les réponses vides valides, notamment le PNG outside-coverage sans appel fournisseur, restent HTTP 200.

Le backend utilise un cache HTTP marin SQLite (http.sqlite3, borne totale de 512 Mio pour les réponses fournisseurs), ainsi qu'un cache bathymétrique SQLite (bathymetry.sqlite3, durée de vie 24 h, 128 Mio disque et 32 Mio mémoire, version marine-visual-7). En production le conteneur API est en lecture seule et le chemin MARINE_CACHE_DIR est monté sur un volume Docker marine-cache ; le cache des fournisseurs OSM doit être disponible en écriture. Aucun cache ni mécanisme de retransmission ne garantit à lui seul la disponibilité des services externes.

Performances Flutter : la PR #126 fusionnée dans dev introduit la réutilisation des états caméra, des encodages GeoJSON identiques, une grille de détection de collision pour de grands ensembles de ports et une reconstruction locale du seul indicateur de cap. Les résultats chiffrés du rapport docs/test-reports/map-smoothness-2026-10-23.md concernent l'hôte WSL/Linux avec Mapbox simulé ; ils ne prouvent pas une amélioration des FPS sur téléphone. Le service HTTP natif Mapbox est limité à deux requêtes simultanées par hôte, toutes sources confondues, indépendamment des deux requêtes GeoJSON Dart. Le rendu natif Mapbox et le trajet réseau doivent être évalués séparément sur appareils réels.

Estimation de position

FastAPI utilise le GPS de l'observateur, l'azimut par rapport au nord vrai, l'inclinaison et la hauteur de la caméra au-dessus du niveau de la mer. Pour le MVP, lorsque le mobile ne dispose pas d'une hauteur mesurée ou calibrée, il utilise par défaut une hauteur de caméra de 2,5 m avec une incertitude de ±0,5 m et camera_height_source = default. Cette valeur constitue un compromis non calibré. L'inclinaison vaut zéro à l'horizon, est positive vers le haut et négative vers le bas. Les métadonnées de focale et de zoom sont conservées lorsqu'elles sont disponibles. Pour le MVP, l'utilisateur place l'objet au centre du viseur avant de déclencher la photo. Le parcours de positionnement utilise la caméra arrière principale. L'ultra-grand-angle, le téléobjectif et la caméra frontale ne sont pas utilisés pour cette estimation. La mise au point privilégie les objets éloignés ou l'infini lorsque l'appareil le permet. Le positionnement photo automatique est destiné aux cibles situées à environ 3 mètres ou plus. L'utilisateur valide toujours le point final.

Le déplacement de la carte ne met pas à jour la position enregistrée de l’appareil. Le rayon de 10 NM s’applique uniquement aux destinataires des alertes.

Alertes, identité et confidentialité

Sélection des destinataires et envoi

Pour un signalement actif nouvellement créé ou modifié, sélectionner les autres utilisateurs actifs ayant activé les alertes de proximité. Leurs appareils doivent être actifs, disposer d’un jeton FCM et d’une position exploitable située à moins de 18 520 m (10 NM). Ni le cap ni l’ancienneté de la dernière position ne filtrent les destinataires.

Enregistrer une entrée alert_history par utilisateur/signalement/version et une tâche notifications par appareil/signalement/version dans la transaction du signalement. L’historique conserve la plus petite distance parmi les appareils éligibles de chaque utilisateur.

Un worker traite les tâches validées et revérifie la version du signalement, sa visibilité, son expiration et l’éligibilité du destinataire. Les erreurs temporaires sont retentées après 1, 5, 15 et 30 minutes, puis toutes les 60 minutes, en respectant un Retry-After plus long et en s’arrêtant à l’expiration. Les échecs permanents passent à l’état failed.

sent signifie que FCM a accepté la requête ; cela ne prouve ni la livraison ni la lecture. L’historique des alertes subsiste même en cas d’échec du push. Les tâches de notification terminales sont supprimées après sept jours.

Contrat FCM mobile

Toutes les valeurs du champ data FCM sont transmises sous forme de chaînes.

Champs obligatoires : alert_id, report_id, report_version, category, position_dms, distance_m et expires_at.

Champs facultatifs : position et alerted_at.

position_dms est la représentation DMS de la position du signalement. position, lorsqu’il est fourni, contient le Point GeoJSON sérialisé.

Si position n’est pas fourni, le mobile reconstruit les coordonnées à partir de position_dms. Si position est fourni, il doit être valide et cohérent avec position_dms ; une incohérence rend le payload invalide.

Le payload FCM sert à identifier et présenter l’alerte. Avant d’afficher le détail ou de cibler la carte, le mobile relit le signalement auprès du backend, qui reste la source de vérité.

Identité et accès

Firebase Authentication gère l’accès par e-mail/mot de passe et les jetons d’identité. Pour l’application mobile, FastAPI vérifie le Firebase ID token, puis contrôle le rôle NoWave, le statut et la propriété de la ressource. Pour le

MVP, ces requêtes mobiles utilisent le contrôle standard du Firebase ID token sans contrôle systématique de révocation à chaque requête. L'administration web constitue une exception volontaire : sa session Firebase est revérifiée avec contrôle de révocation à chaque requête administrative, puis le profil NoWave est relu afin de vérifier que le compte est toujours actif et possède toujours le rôle admin. Les mots de passe ne sont pas stockés dans PostgreSQL. Un utilisateur dont le statut est suspended ne peut plus accéder aux fonctionnalités de l'application tant que la suspension est active. Après authentification, le mobile peut uniquement récupérer son profil afin d'identifier la suspension et effectuer les opérations nécessaires à la déconnexion, au nettoyage de l'appareil et, le cas échéant, à la suppression de son propre compte. Les routes métier refusent ses requêtes avec HTTP 403 et le code user_inactive. GET /users/me reste accessible pour permettre au mobile de connaître le statut du compte. DELETE /devices/current reste accessible pour permettre le nettoyage de l'appareil avant la fermeture de la session. La sélection des destinataires d'alertes continue d'exclure les utilisateurs suspendus.

Visibilité du profil et confidentialité

show_user_name et show_boat_info valent false par défaut. notifications_enabled est une préférence distincte qui vaut également false par défaut. La date de naissance et la nationalité restent privées. Les détails d'un signalement n'exposent les informations sur l'auteur et le bateau que si la préférence correspondante l'autorise.

La suppression d'un compte retire les données personnelles et les appareils tout en conservant les signalements avec author_id = NULL. Ces signalements affichent « Utilisateur supprimé ». Lors d'un changement de compte, l'application efface l'ancienne session et la position de l'appareil, puis enregistre un nouvel UUID d'installation.

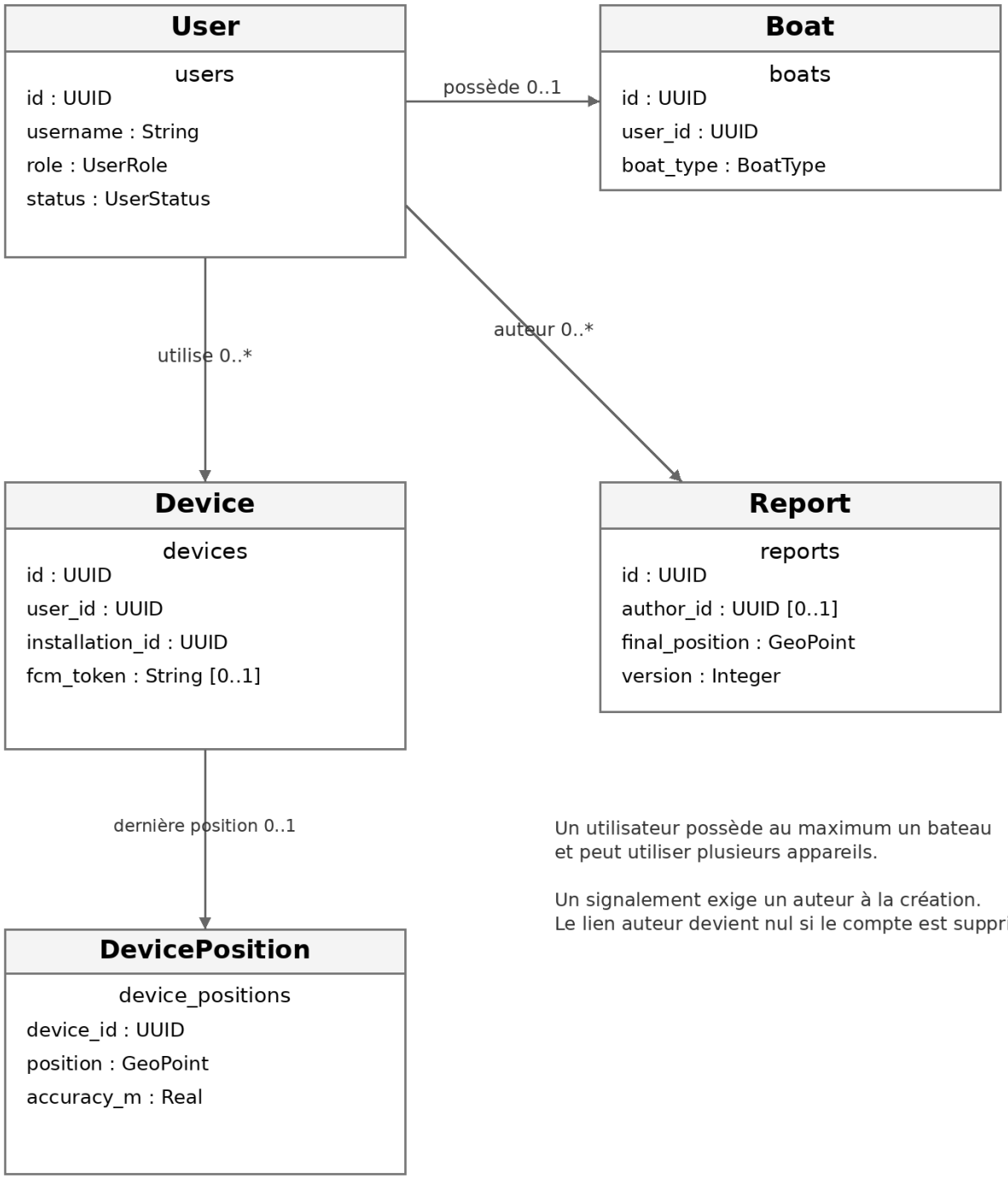
Déploiement

Environnement	Organisation
Développement	Docker Compose exécute FastAPI, PostgreSQL/PostGIS et Nginx avec les Dockerfiles du déploiement. Le site et /admin sont accessibles sur https://localhost:8443 ; HTTP :8080 redirige vers HTTPS. Le certificat local auto-signé est conservé dans un volume. Le backend reste accessible en boucle locale sur :8000 et applique les migrations au démarrage. Les secrets de développement sont requis ; Flutter s'exécute localement.
Test	Environnement de test et base de données séparés.
Production	Un VPS Hostinger héberge les conteneurs FastAPI, PostgreSQL/PostGIS et Nginx. PostgreSQL reste sur le réseau privé Docker et n'est pas exposé directement sur Internet. Cloudflare protège les services publics. Les secrets sont configurés hors de Git. Pour l'administration web, la clé Firebase Web API est injectée dans le conteneur FastAPI via FIREBASE_WEB_API_KEY_FILE=/run/secrets/firebase_web_api_key ; le fichier source est conservé sur le VPS dans /opt/nowave/secrets/firebase_web_api_key et ne doit jamais être committé.
Contrôles de santé	/health contrôle FastAPI, /health/ready contrôle l'accès à la base de données.

Automatisation GitHub vérifiée sur dev : checks.yml exécute les contrôles Python, Flutter, PostGIS, Web et Admin Web ; PostGIS inclut les parcours administrateur Playwright avec une base isolée et les assets web compilés. production.yml lance ensuite les audits de dépendances et d'images. Son job de déploiement possède la condition github.event_name == push et github.ref == refs/heads/dev, et dépend de la validation. La variable NOWAVE_PRODUCTION_DEPLOY_ENABLED ne figure pas dans le workflow actuel : toute instruction documentaire qui la présenterait comme un verrou actif est obsolète. Le job utilise l'environnement GitHub production ; les éventuels approbateurs et règles de protection configurés sur GitHub doivent être contrôlés séparément. La version effectivement disponible sur le VPS doit être vérifiée avec GET /version et les logs du workflow. Le frontal doit livrer les assets du même build que les pages admin : /assets/admin-map.js et /assets/nowave.css sont configurés avec expires -1 en production pour imposer leur revalidation.

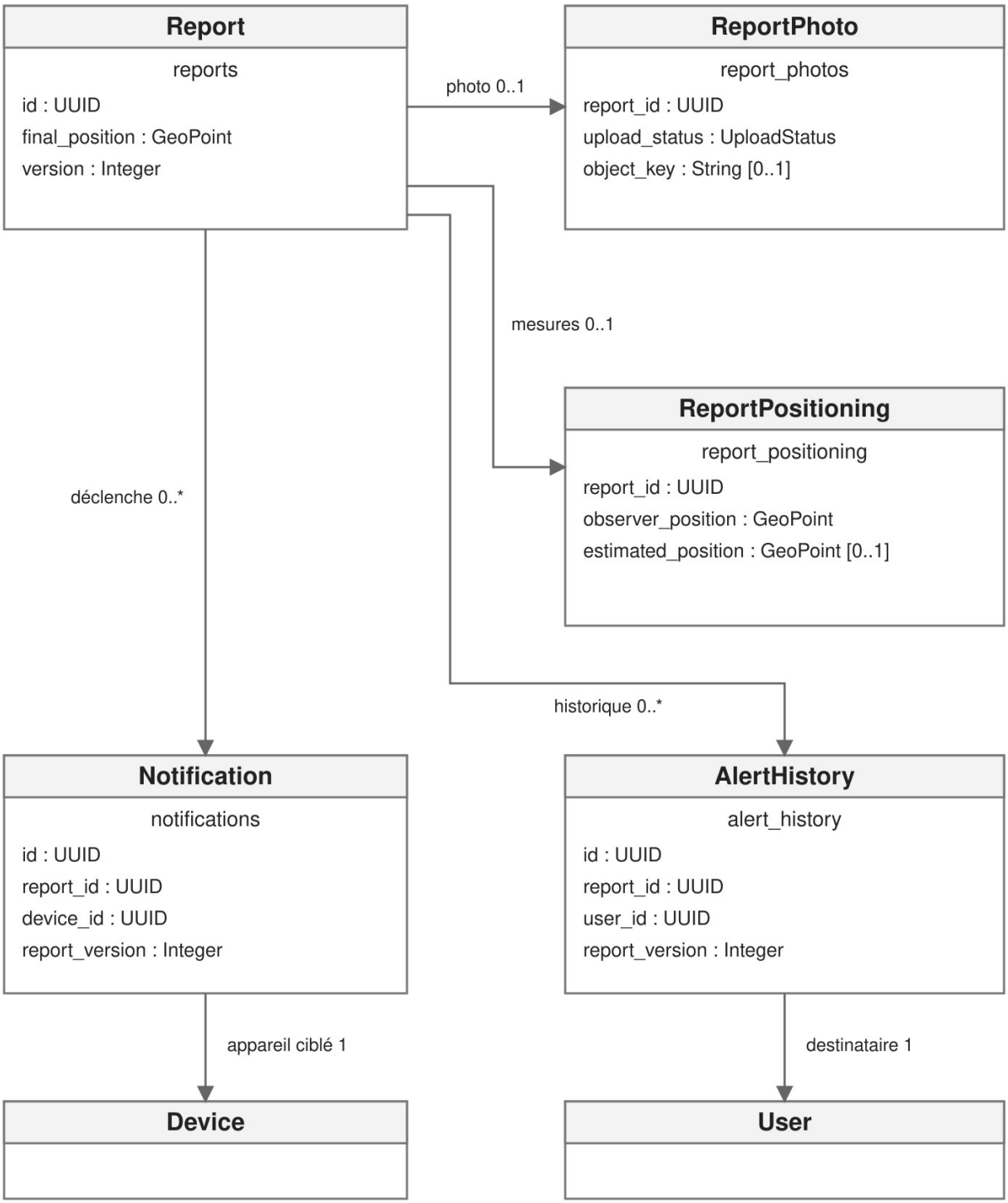
UML comptes, appareils et signalements

Les vues de classes présentent les principaux champs et toutes les relations. Le dictionnaire des 10 tables métier et de leurs 99 champs suit. La table technique map_cache_revision et ses deux champs sont également décrits. Les boîtes de classe répétées désignent les mêmes entités.



1 = exactement un, 0..1 = lien unique optionnel, 0..* = zéro ou plusieurs. GeoPoint utilise les coordonnées WGS-84.

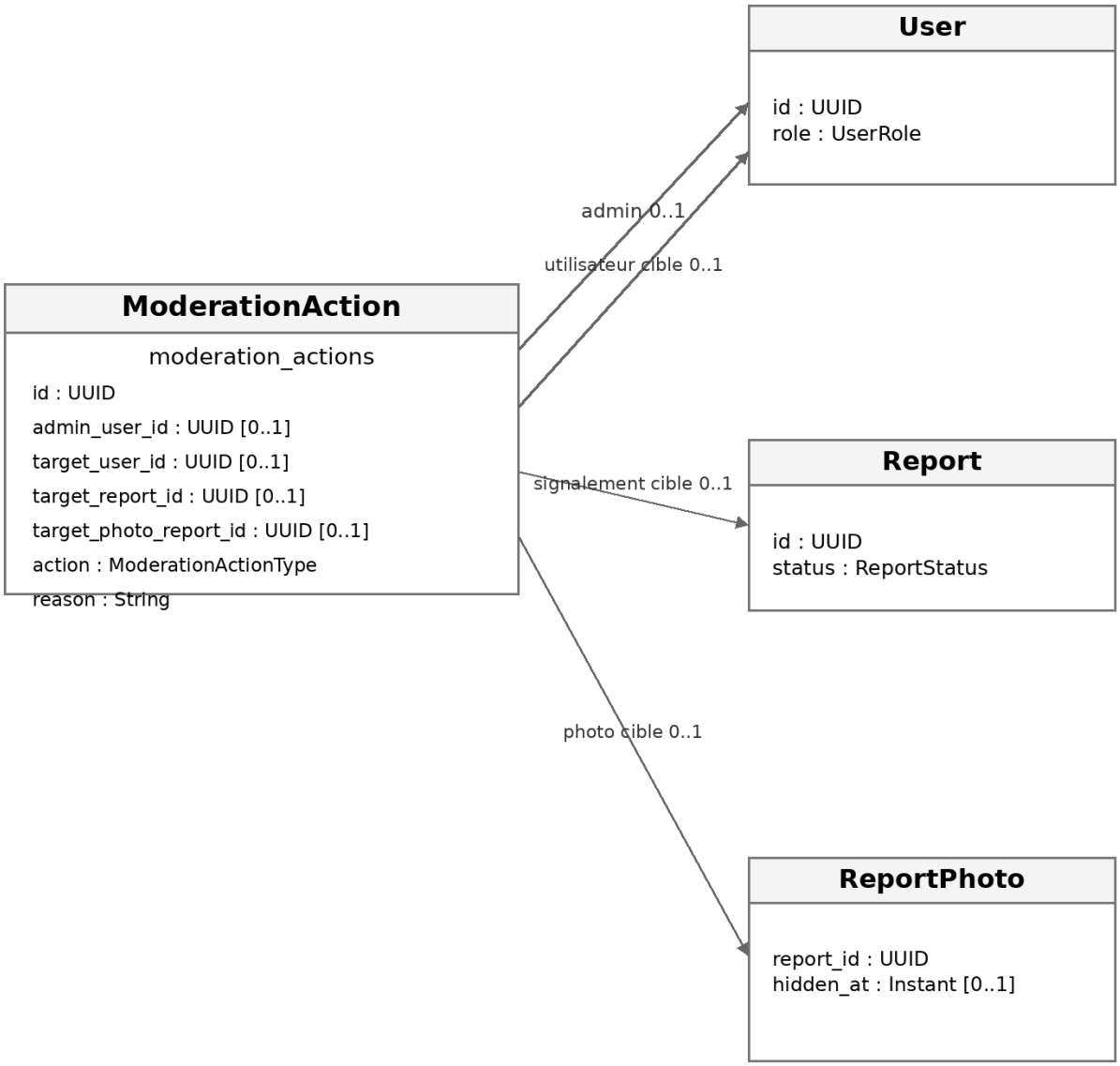
UML signalements photo et alertes



Le mode photo possède une ligne de positionnement et une ligne photo ;
le mode manuel n'en possède aucune. L'historique est par utilisateur,
les tâches d'envoi sont par appareil.

Le mode photo possède une ligne de positionnement et une ligne photo ; le mode manuel n'en possède aucune. L'historique des alertes est géré par utilisateur, les tâches d'envoi par appareil. report_version stocke un nombre, et non une clé étrangère vers reports.version.

UML modération et audit



Un administrateur est un User ayant le rôle admin. Chaque nouvelle entrée d'audit enregistre l'administrateur ayant réalisé l'action, une cible correspondant à cette action et un motif non vide. L'action et son enregistrement d'audit sont sauvegardés ensemble.

Les liens d'audit utilisent SET NULL lorsqu'un compte ou une cible est supprimé. L'audit reste disponible, tandis que le lien vers l'entité supprimée peut être absent.

Base de données utilisateurs et bateau

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

users

Profil utilisateur, rôle, statut et préférences.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant interne.
firebase_uid	text	UQ, NN	Identifiant Firebase Authentication.
username	varchar(100)	UQ, NN	Nom unique, affiché selon show_user_name, y compris dans les observations publiques. La modération possède un accès distinct.
email	varchar(254)	UQ, NN	E-mail privé du compte, issu de l'identité Firebase vérifiée.
date_of_birth	date	NULL	Date de naissance privée.
nationality	char(2)	NULL	Code pays ISO privé.
role	varchar(10)	NN	user - admin.
status	varchar(10)	NN	active - suspended.
show_user_name	boolean	NN, default false	Consentement à l'affichage du nom d'utilisateur.
show_boat_info	boolean	NN, default false	Consentement à l'affichage des informations du bateau.
notifications_enabled	boolean	NN, default false	Préférence NoWave pour les alertes de proximité.
created_at	timestamptz	NN	Date de création.
updated_at	timestamptz	NN	Date de dernière mise à jour.

boats

Un bateau optionnel au maximum par utilisateur.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant du bateau.
user_id	uuid	FK users.id, UQ, NN	Propriétaire ; un bateau au maximum par utilisateur.
name	varchar(100)	NULL	Nom du bateau optionnel.
boat_type	varchar(20)	NN	voilier - bateau_moteur - catamaran - semi_rigide - jet_ski - autre.
flag_country	char(2)	NULL	Pays du pavillon.
created_at	timestamptz	NN	Date de création.
updated_at	timestamptz	NN	Date de dernière mise à jour.

Base de données appareils et position

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

devices

Installation mobile et jeton FCM.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant de l'installation.
user_id	uuid	FK users.id, NN	Utilisateur de l'appareil.
installation_id	uuid	UQ, NN	Identifiant généré par l'application.
fcm_token	text	UQ, NULL	Jeton FCM, NULL lorsqu'il est indisponible ou révoqué.
platform	varchar(10)	NN	android - ios.
is_active	boolean	NN	Appareil actif pour la sélection des alertes.
last_seen_at	timestamptz	NN	Dernière activité connue.

device_positions

Dernière position exploitable de l'appareil, sans historique de positions.

Champ	Type SQL	Contraintes	Signification
device_id	uuid	PK/FK devices.id	Une position courante par appareil.
position	geography (Point,4326)	NN	Position GPS.
accuracy_m	double precision	NN, CHECK 0 <= x <= 50	Précision GPS requise pour une position stockée exploitable.
heading_deg	double precision	NULL, CHECK 0<=x<360	Cap en degrés ; oriente l'icône et ne filtre jamais les alertes.
measured_at	timestamptz	NN	Heure de mesure du téléphone. Une avance maximale de 5 minutes par rapport à l'heure courante du serveur est tolérée ; au-delà, la mesure est rejetée.
received_at	timestamptz	NN	Heure de réception par le serveur.

Mises à jour de position

Un appareil conserve uniquement sa dernière position exploitable et aucun historique de navigation. Les mesures plus récentes remplacent les anciennes. La validation de la source exige également une précision GPS finie comprise entre 0 et 50 m. measured_at peut être au maximum 5 minutes dans le futur par rapport à l'heure courante du serveur ; au-delà de cette tolérance, la mesure est rejetée. heading_deg modifie uniquement l'orientation de l'icône.

Lors d'un changement de compte, conserver le propriétaire de l'ancienne ligne device. Effacer son jeton, supprimer l'ancienne position et les tâches en file d'attente dans une même transaction, se coordonner avec

NotificationWorker, effacer la session de l'application et enregistrer un nouvel UUID d'installation.

Base de données signalements

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

reports

Signalement maritime et position finale confirmée.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant du signalement.
author_id	uuid	FK users.id, NULL	Auteur obligatoire à la création, NULL après suppression du compte.
client_report_id	uuid	NN	Identifiant client utilisé pour l'idempotence et la reprise après une interruption réseau.
category	varchar(20)	NN	marine_animal - obstruction - pollution.
positioning_mode	varchar(10)	NN	photo - manual.
description	varchar(250)	NULL	Texte libre, jusqu'à 250 caractères.
final_position	geography (Point,4326)	NN	Position publiée confirmée ou corrigée par l'utilisateur.
observed_at	timestamptz	NN	Heure réelle d'observation.
created_at	timestamptz	NN	Heure de création serveur.
expires_at	timestamptz	NN	observed_at + 24 h.
status	varchar(10)	NN	active - removed - expired.
version	integer	NN, default 1	Version initiale à 1. La modération ne l'incrémente pas. L'édition par l'auteur n'est pas livrée dans le MVP.
updated_at	timestamptz	NN	Date de dernière mise à jour.
removed_at	timestamptz	NULL	Date de retrait par la modération. Le retrait par l'auteur n'est pas livré dans le MVP.

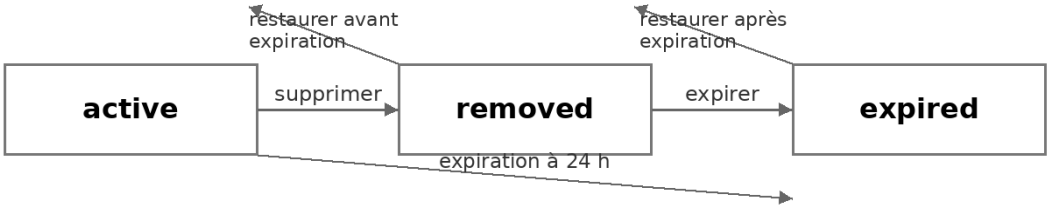
Identité et durée de vie

UNIQUE(author_id, client_report_id) garantit l'idempotence de la création d'un signalement et évite les doublons lors d'une répétition de requête après une interruption réseau. L'auteur est obligatoire à la création. La suppression du compte met author_id à NULL et conserve le signalement.

expires_at = observed_at + 24 heures. Côté serveur, un signalement est exclu des tuiles dès que expires_at <= now() ou que status n'est plus active ; aucun job d'expiration n'est requis. CON-024 (option A) : les réponses MVT sont mises en cache côté client avec Cache-Control: private, max-age=15, must-revalidate. Une tuile précédemment reçue peut donc afficher temporairement un signalement expiré ; le TTL de 15 secondes ne garantit

pas à lui seul la disparition du repère dans Flutter. Une restauration administrative conserve l'expiration d'origine. Aucun changement de code ni réduction de cache n'est décidé.

États effectifs d'un signalement



La transition de 24 heures dans cette vue représente un contrôle effectif d'expiration, et non une mise à jour obligatoire de la base. Une ligne peut conserver status = active après expires_at ; les requêtes cartographiques l'excluent malgré tout. L'historique affiche expired lorsque la durée de vie est écoulée, sauf si le signalement a été supprimé.

Base de données mesures de positionnement photo

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

report_positioning

Mesures utilisées pour le positionnement par photo.

Champ	Type SQL	Contraintes	Signification
report_id	uuid	PK/FK reports.id, NN	Au maximum une ligne par signalement photo.
observer_position	geography (Point,4326)	NN	Position du téléphone au moment de la capture.
gps_accuracy_m	double precision	NN, CHECK 0 <= x <= 50	Précision GPS au moment de la capture.
azimuth_deg	double precision	NN, CHECK 0<=x<360	Azimut par rapport au nord vrai.
inclination_deg	double precision	NN	0 degré à l'horizon, positif vers le haut, négatif vers le bas.
camera_height_m	numeric(6,2)	NN, >0	Hauteur du téléphone au-dessus du niveau de la mer. Le mobile MVP utilise 2,5 m lorsque camera_height_source = default.
camera_height_source	varchar(30)	NULL	Source de la hauteur. Le mobile MVP utilise default pour la valeur non calibrée.
camera_height_uncertainty_m	double precision	NULL	Incertitude sur la hauteur. Le mobile MVP utilise 0,5 m lorsque camera_height_source = default.
focal_length_mm	double precision	NULL	Longueur focale à la capture.

Champ	Type SQL	Contraintes	Signification
zoom_ratio	double precision	NULL	Niveau de zoom utilisé.
estimated_position	geography (Point,4326)	NULL	Position calculée automatiquement lorsqu'elle est exploitable.
estimated_distance_m	double precision	NULL	Distance estimée de l'objet.
algorithm_version	varchar(50)	NULL	Version de l'algorithme de calcul.
captured_at	timestamptz	NN	Heure de capture.

Conventions de mesure

GeoPoint correspond à geography(Point,4326) et utilise WGS-84. L'azimut est mesuré par rapport au nord vrai. L'inclinaison vaut zéro à l'horizon, est positive vers le haut et négative vers le bas. La hauteur de la caméra est mesurée au-dessus du niveau de la mer. Les mesures de positionnement et les métadonnées caméra disponibles correspondent au moment exact de la capture et ne sont pas remplacées par des valeurs mesurées après le déclenchement.

Ces mesures n'appartiennent qu'au mode photo. Pour le MVP, la cible est supposée placée au centre du viseur au moment de la capture. Aucune sélection de l'objet dans l'image n'est requise après la prise de photo. L'utilisateur confirme ou corrige final_position même lorsqu'une estimation existe. Si le calcul échoue, estimated_position reste absent et le signalement demeure en mode photo.

Base de données stockage des photos

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

report_photos

Métadonnées photo pour un signalement photo.

Champ	Type SQL	Contraintes	Signification
report_id	uuid	PK/FK reports.id, NN	Photo unique d'un signalement photo.
upload_status	varchar(10)	NN	pending - uploaded - failed.
object_key	text	UQ, NULL	Clé d'objet Scaleway après envoi.
size_bytes	integer	NULL, CHECK 1 <= x <= 500000	Taille maximale du fichier : 500 000 octets.
mime_type	varchar(100)	NULL, CHECK image/jpeg	JPEG uniquement.
uploaded_at	timestamptz	NULL	Heure d'envoi réussi.
hidden_at	timestamptz	NULL	Heure à laquelle la photo a été masquée par la modération.
created_at	timestamptz	NN	Heure de création de la ligne.
updated_at	timestamptz	NN	Date de dernière mise à jour.

États d'envoi

État	Signification
pending	Ligne photo créée côté serveur, sans envoi confirmé. Un JPEG temporaire peut subsister dans le parcours mobile ouvert.
uploaded	Le JPEG a été envoyé et ses métadonnées de stockage ont été enregistrées.
failed	L'envoi s'est terminé par un échec.

Les photos sont stockées dans Scaleway ; PostgreSQL ne conserve que les métadonnées. Chaque signalement photo possède exactement une ligne report_photos, tandis que les signalements manuels n'en ont aucune. Un échec d'envoi n'invalide pas le signalement. Réessayer avec le même report_id et la même ligne photo ; en cas de succès, définir upload_status = uploaded. Ne jamais insérer une seconde ligne photo.

FastAPI accepte un JPEG non vide et décodable, d'au plus 500 000 octets, sans EXIF. Une photo masquée conserve son état d'envoi et est exclue de l'affichage normal ainsi que des nouvelles URL de lecture générées.

Stockage objet et nettoyage

Le bucket Scaleway Object Storage est privé. Les objets photo utilisent le préfixe reports/{report_id}/{sha256}/{attempt_uuid}.jpg afin de distinguer chaque tentative. Les uploads et le nettoyage sérialisent l'accès à la ligne report_photos avec un verrou PostgreSQL jusqu'au commit. Les photos référencées sont conservées, y compris lorsqu'elles sont masquées ou que le signalement a expiré.

Le worker serveur photo_cleanup rescane le bucket toutes les 5 minutes et supprime uniquement les objets non référencés âgés d'au moins 1 heure. Un échec de suppression est journalisé puis retenté au passage suivant. Le bucket utilisé pour cette politique de purge est privé et sans versioning. En développement, le worker peut être activé avec docker compose --profile photos up -d ; en production, il est démarré avec l'API.

Base de données notifications et historique

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

notifications

File persistante d'envoi FCM par version de signalement.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant de la tâche d'envoi.
report_id	uuid	FK reports.id, NN	Signalement associé.
device_id	uuid	FK devices.id, NN	Appareil destinataire.
report_version	integer	NN	Version du signalement ayant déclenché l'alerte.
status	varchar(10)	NN	pending - sent - failed.
attempt_count	integer	NN, default 0	Nombre de tentatives FCM.
next_attempt_at	timestamptz	NULL	Prochaine tentative lorsque la tâche est pending.

Champ	Type SQL	Contraintes	Signification
sent_at	timestamptz	NULL	Heure à laquelle FCM accepte la requête d'envoi.
last_error_code	text	NULL	Dernière erreur ou cause terminale.
created_at	timestamptz	NN	Heure de création de la tâche.
updated_at	timestamptz	NN	Heure du dernier changement d'état.

alert_history

Historique des alertes utilisateur par version de signalement, avec la distance minimale parmi les appareils éligibles.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant de l'entrée d'historique.
user_id	uuid	FK users.id, NN	Utilisateur recevant l'alerte.
report_id	uuid	FK reports.id, NN	Signalement associé.
report_version	integer	NN	Version du signalement ayant déclenché l'alerte.
category	varchar(20)	NN	Instantané de la catégorie du signalement.
position	geography (Point,4326)	NN	Instantané de la position finale du signalement.
distance_m	double precision	NN, CHECK >= 0	Distance minimale entre le signalement et les appareils éligibles de cet utilisateur lors de la sélection des destinataires.
alerted_at	timestamptz	NN	Heure de création de l'alerte.

Base de données actions de modération

PK = clé primaire, FK = clé étrangère, UQ = unique, NN = non nul. NULL indique une valeur optionnelle.

moderation_actions

Journal d'audit des actions d'administration et de modération.

Champ	Type SQL	Contraintes	Signification
id	uuid	PK, NN	Identifiant de l'action.
admin_user_id	uuid	FK users.id, NULL	Administrateur ; SET NULL après suppression du compte.
target_user_id	uuid	FK users.id, NULL	Utilisateur cible, le cas échéant.
target_report_id	uuid	FK reports.id, NULL	Signalement cible, le cas échéant.
target_photo_report_id	uuid	FK report_photos.report_id, NULL	Photo cible, le cas échéant.

Champ	Type SQL	Contraintes	Signification
action	varchar(20)	NN	remove_report - restore_report - hide_photo - suspend_user - reactivate_user.
reason	text	NN	Motif de l'action.
previous_status	varchar(10)	NULL	Statut précédent, le cas échéant.
new_status	varchar(10)	NULL	Nouveau statut, le cas échéant.
created_at	timestamptz	NN	Heure de l'action.

Action et cible

Action	Cible
remove_report	Report
restore_report	Report
hide_photo	ReportPhoto
suspend_user	User
reactivate_user	User

À la création, une entrée d'audit identifie une cible correspondant à l'action ainsi qu'un administrateur actif. Son contenu ne peut pas être modifié. La suppression ultérieure d'un parent peut effacer la clé étrangère correspondante.

La suspension d'un utilisateur bloque les futures publications et alertes, mais ne supprime pas automatiquement les signalements existants. Un administrateur supprime un signalement séparément.

Table technique map_cache_revision

La migration 20261012_0003 ajoute cette table au schéma nowave. Elle complète les 10 tables métier, soit 11 tables et 101 champs décrits. Une ligne unique porte la révision des signalements. Un trigger renouvelle cet UUID après INSERT, UPDATE, DELETE ou TRUNCATE sur reports afin d'invalider le cache des tuiles après validation de la transaction.

Champ	Type SQL	Contraintes	Signification
singleton	boolean	PK, default true, CHECK true	Clé de la ligne unique de révision.
revision	uuid	NN, default gen_random_uuid()	UUID renouvelé à chaque instruction modifiant reports.

Intégrité de la base de données et valeurs partagées

Règle	Spécification
Idempotence de création	UNIQUE(author_id, client_report_id) sur reports.
Doublons d'envoi	UNIQUE(report_id, device_id, report_version) sur notifications.
Doublons d'historique	UNIQUE(user_id, report_id, report_version) sur alert_history.

Règle	Spécification
Cohérence photo	Le mode photo crée report_positioning et report_photos avec son signalement. Le mode manuel n'en crée aucun. Enregistrer les lignes associées dans la même transaction.
Index spatiaux	GiST sur reports.final_position et device_positions.position.
Données cartographiques	Les tuiles MVT et les clusters d'affichage sont générés à partir de reports. Ce ne sont pas des tables persistantes.
Suppression d'un parent	Les lignes dépendantes de bateau, appareil, position, photo, positionnement, notification et historique destinataire suivent la suppression de leur parent. Les auteurs des signalements et les liens d'audit utilisent SET NULL.
Distance d'alerte	PostGIS sélectionne les appareils à moins de 18 520 m. L'instantané de l'historique conserve la distance minimale parmi les appareils éligibles de l'utilisateur.

Valeurs stockées

Type	Valeurs
UserRole	user, admin
UserStatus	active, suspended
Platform	android, ios
PositioningMode	manual, photo
ReportStatus	active, removed, expired
UploadStatus	pending, uploaded, failed
ReportCategory	marine_animal, obstruction, pollution
NotificationStatus	pending, sent, failed
BoatType	voilier, bateau_moteur, catamaran, semi_rigide, jet_ski, autre

UUID identifie un enregistrement. Instant correspond à timestamptz. L'API échange les horodatages en UTC et les points GeoJSON avec la longitude avant la latitude. ModerationActionType utilise les cinq actions listées dans la section Base de données : actions de modération.

Règles d'intégrité supplémentaires

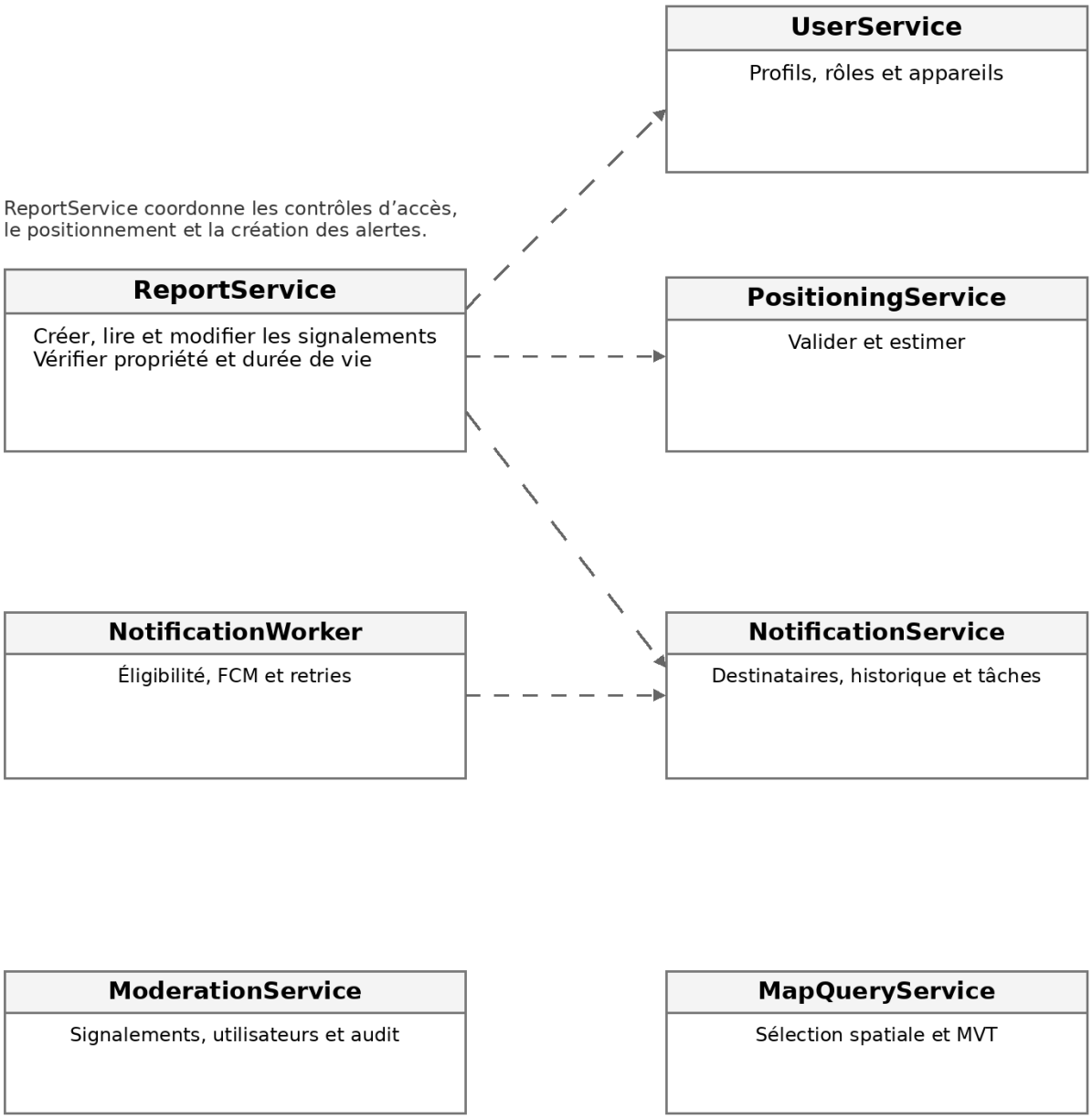
Les mesures doivent être finies lorsqu'elles sont présentes. Les deux champs de précision GPS sont compris entre 0 et 50 m. Pour les valeurs optionnelles non nulles : l'incertitude de hauteur et la distance estimée sont >= 0, la focale et le zoom sont > 0. Report.version et les deux champs report_version sont >= 1, attempt_count est >= 0, et la taille de la photo est > 0 et <= 500000 lorsqu'elle est présente.

Des triggers de contraintes différées vérifient le mode photo et ses deux lignes enfants au moment du commit. Les notifications pending exigent next_attempt_at, les notifications sent exigent sent_at.

Des index supplémentaires couvrent reports.author_id, reports.status/expires_at, devices.user_id et notifications.status/next_attempt_at.

UML services et dépendances

Ces services appartiennent au même backend FastAPI. Les flèches en pointillés représentent une dépendance. Le worker traite les tâches de notification après validation de la transaction du signalement. Un worker serveur distinct, photo_cleanup, entretient le stockage objet des photos sans modifier la durée de vie des signalements.



MapQueryService lit les signalements actifs dans la tuile demandée via PostgreSQL/PostGIS. Il est séparé de la publication des signalements et de la sélection des destinataires. ModerationService gère la suppression/restauration des signalements, le masquage des photos et la suspension/réactivation des comptes.

Principales méthodes des services

Le diagramme de services est complété par l'inventaire des méthodes ci-dessous. Les attributs et relations du domaine sont définis dans les vues de classes et dans le dictionnaire des 10 tables métier et de leurs 99 champs, complété par les deux champs de la table technique map_cache_revision. Les services coordonnent ces entités au lieu de dupliquer leurs champs stockés.

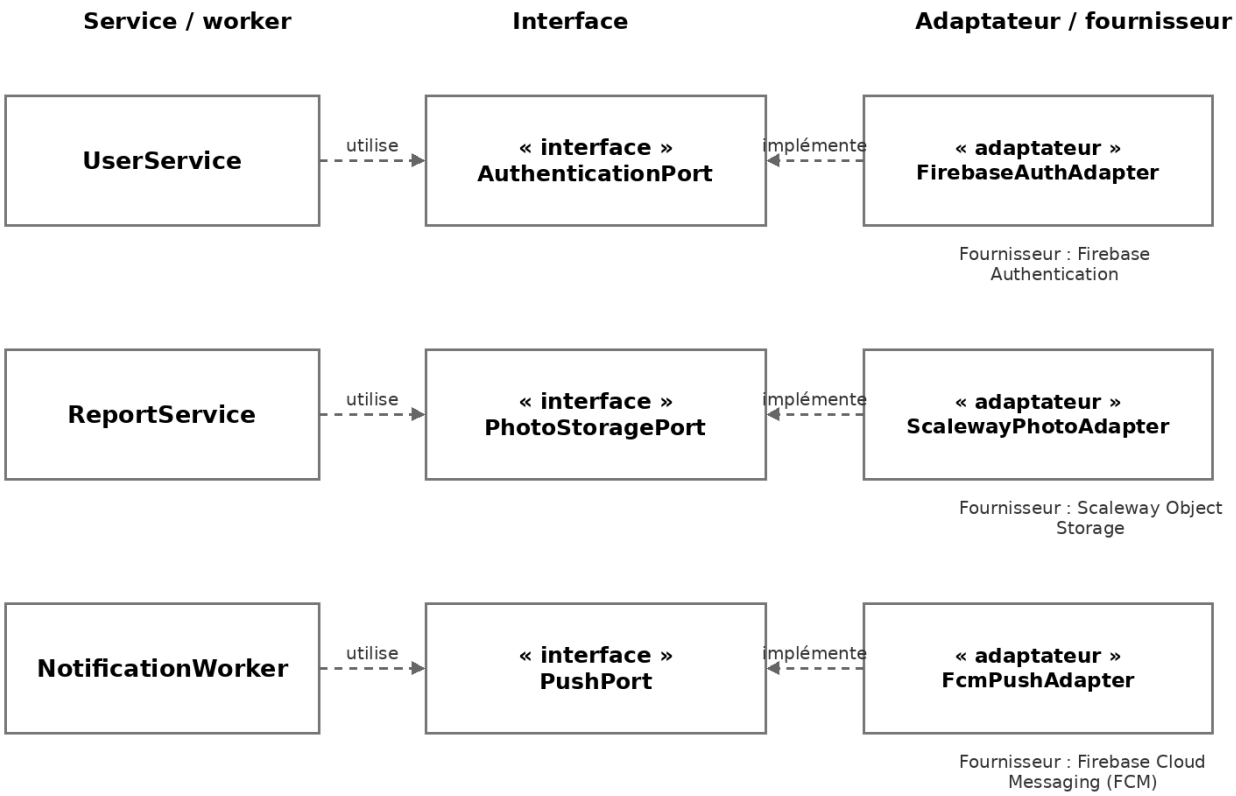
Service	Méthodes principales	Responsabilité
UserService	authenticate(), updateProfile(), registerDevice(), updatePreferences(), recordPosition()	Contrôles d'identité, profils, consentement, enregistrement des appareils et dernière position exploitable. La création initiale du profil est exposée via POST /users/me.
ReportService	createReport()	Publication, idempotence et expiration par horloge serveur. Aucune route de modification/retrait par auteur dans le MVP.
PositioningService	estimate(), validateFinalPosition()	Calculer une estimation photo et valider le point final confirmé par l'utilisateur.
NotificationService	selectTargets(), createHistory(), createTasks(), isEligible()	Sélectionner les destinataires, enregistrer une entrée d'historique par utilisateur/version et une tâche par appareil/version, puis revérifier l'éligibilité.
ModerationService	removeReport(), restoreReport(), hidePhoto(), suspendUser(), reactivateUser()	Les cinq méthodes exigent reason. Appliquer atomiquement chaque changement d'état et l'entrée d'audit immuable.

Worker et requêtes cartographiques

NotificationWorker traite les tâches validées, contrôle l'éligibilité et l'expiration, envoie via PushPort et enregistre les nouvelles tentatives ou les résultats terminaux. MapQueryService lit uniquement les signalements actifs et non expirés dans la tuile demandée via PostGIS. La requête cartographique et le worker évaluent tous deux expires_at par rapport à l'heure courante du serveur.

UML interfaces externes

Les ports séparent la logique métier des fournisseurs. Les diagrammes conservent les interfaces et adaptateurs nommés définis dans l'UML fourni, avec leurs responsabilités documentées.



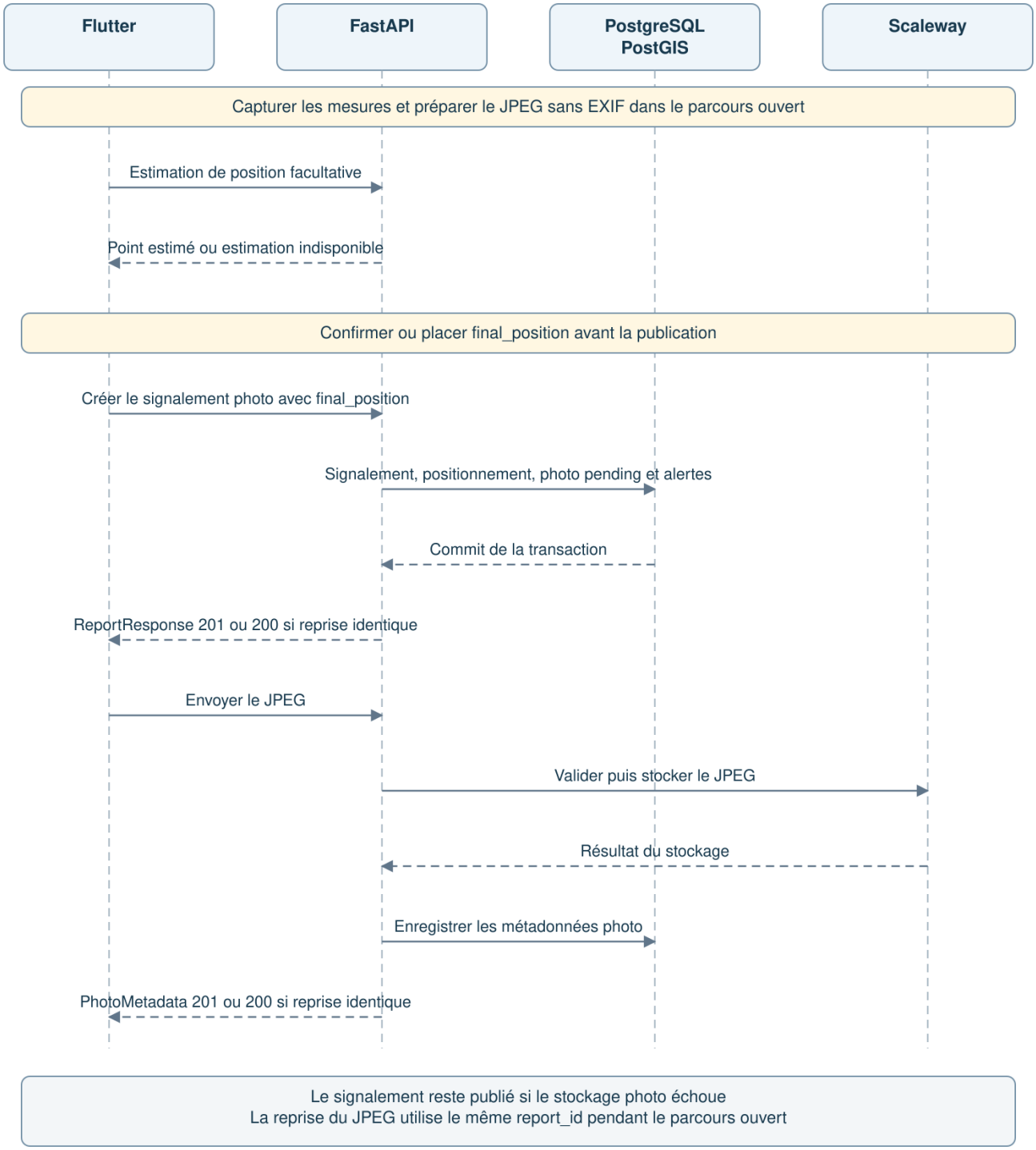
Interface	Responsabilité
AuthenticationPort	Vérifier l'identité Firebase.
PhotoStoragePort	Envoyer les objets JPEG et fournir des URL de lecture signées temporaires.
PushPort	Envoyer les requêtes de notification via FCM.

Lire chaque ligne du service vers son interface. L'adaptateur implémente cette interface ; le triangle creux pointe vers l'interface. Le fournisseur est indiqué sous l'adaptateur.

Ouvrir le document UML complet

Séquence 1 publier un signalement photo

L'auteur est authentifié et actif. POST /api/v1/position-estimates calcule le point. POST /api/v1/reports crée le signalement ; POST /api/v1/reports/{report_id}/photo envoie le JPEG séparément.

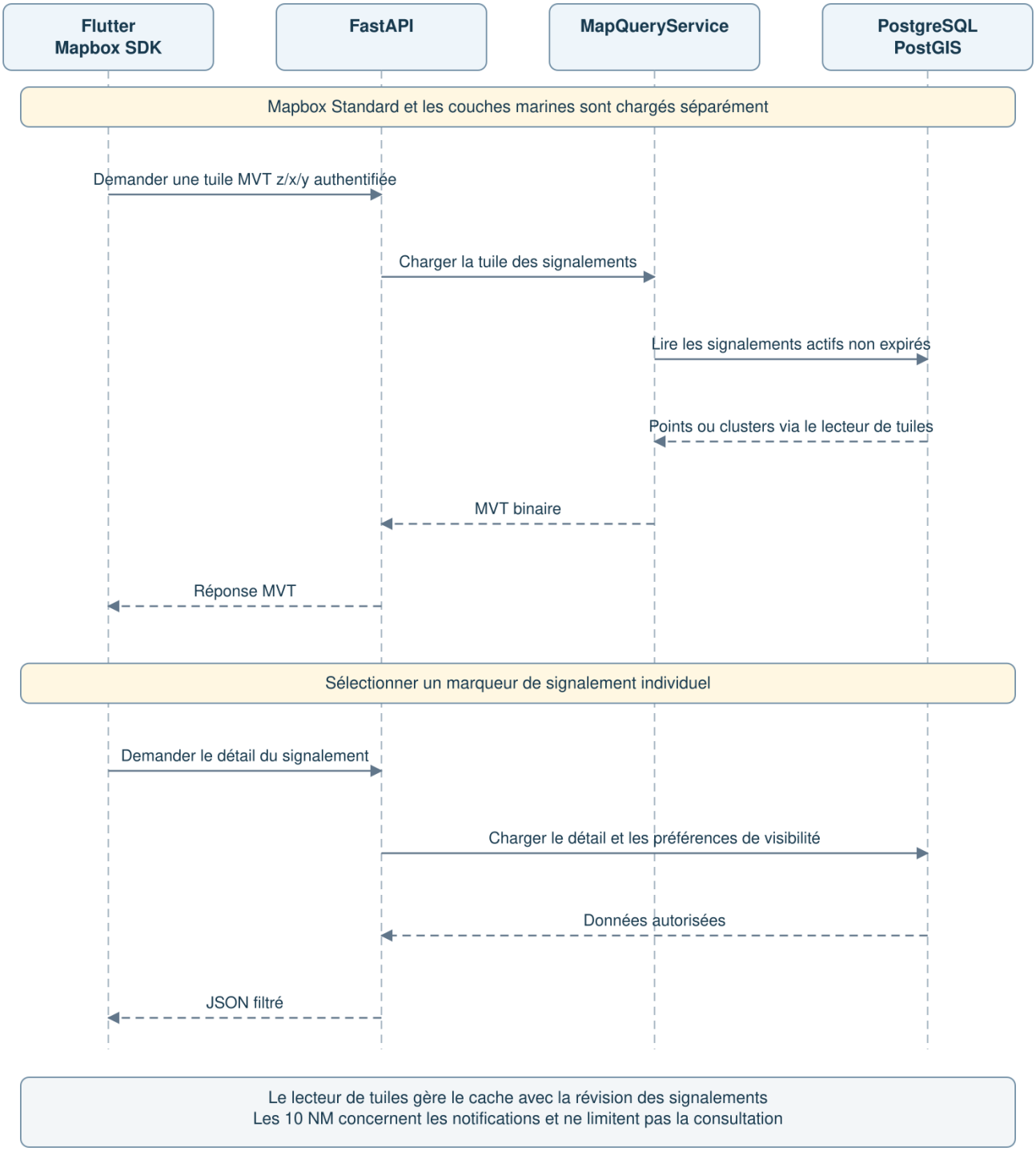


Flèches pleines : requêtes. Flèches en pointillés : réponses. Lire les échanges de haut en bas.

Un client_report_id répété renvoie le signalement déjà stocké sans créer une seconde vague d'alertes. Si l'estimation échoue, le signalement reste en mode photo. Un échec de l'envoi du JPEG ne remet pas en cause la publication du signalement. Tant que le parcours de publication reste ouvert, le JPEG peut être conservé temporairement afin de permettre une nouvelle tentative avec le même report_id et la même ligne report_photos. Si le parcours est fermé avant un envoi réussi, le signalement reste actif mais aucune photo n'est disponible et aucune reprise automatique de l'envoi n'est prévue.

Séquence 2 consulter la carte

GET /api/v1/map/tiles/{z}/{x}/{y}.mvt charge la zone visible. GET /api/v1/reports/{report_id} charge les détails complets lors de la sélection via ReportService. Ces lectures mobiles des signalements transportent le jeton Firebase. Les routes marines et les lectures de la carte publique web décrites dans l'index sont accessibles sans authentification.

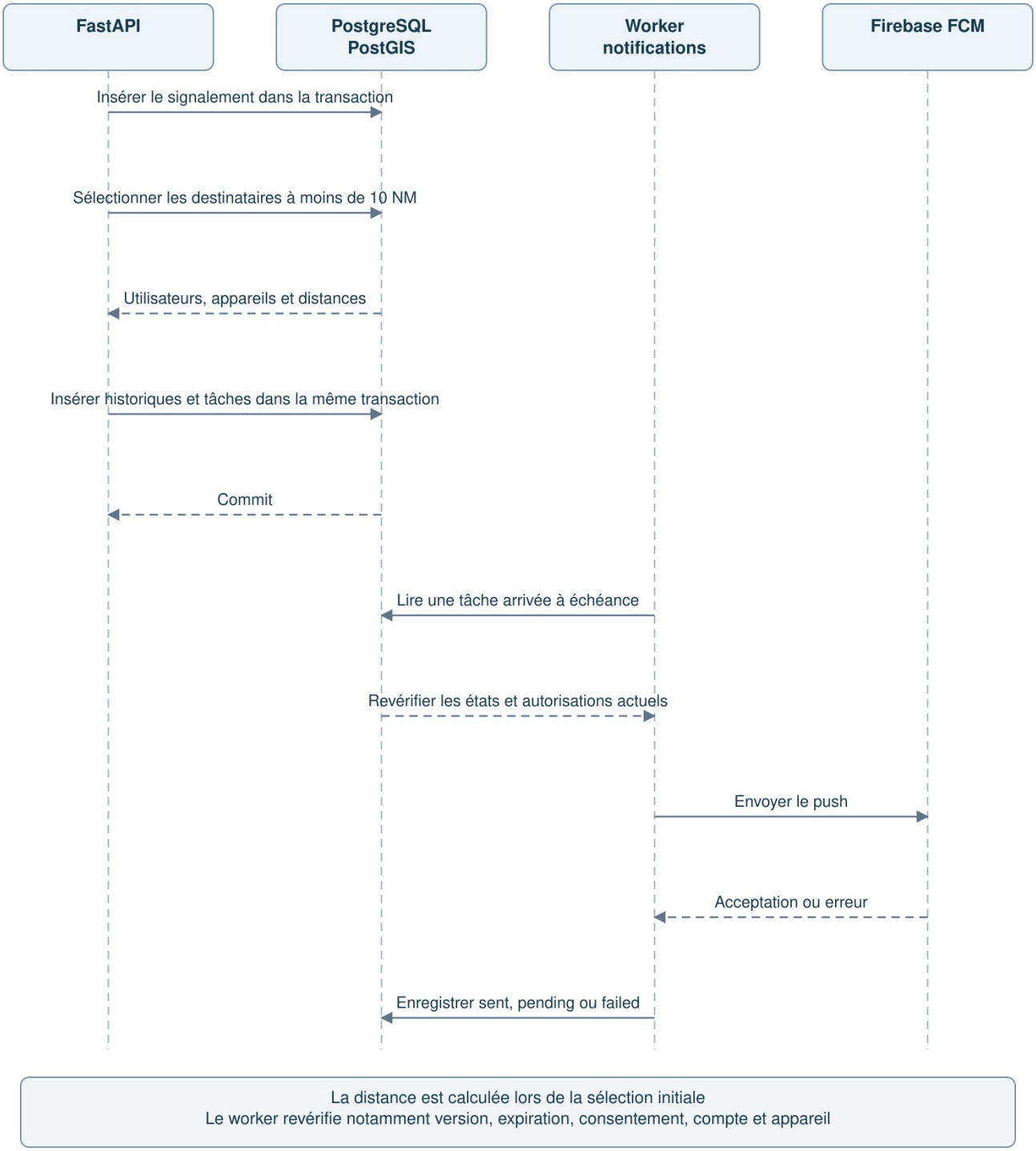


Flèches pleines : requêtes. Flèches en pointillés : réponses. Lire les échanges de haut en bas.

Répéter les échanges de tuiles pour la zone visible lorsque l'application est connectée. Le déplacement de la carte ne modifie ni le GPS de l'appareil ni les destinataires des alertes. Les détails du signalement vérifient les droits d'accès actuels et la visibilité de la photo.

Séquence 3 alerter les utilisateurs à proximité

La sélection des destinataires et la création des tâches ont lieu dans la transaction du signalement. Un worker traite les tâches validées. L'historique est unique par utilisateur/signalement/version ; les tâches d'envoi sont uniques par appareil/signalement/version.



Flèches pleines : requêtes. Flèches en pointillés : réponses. Lire les échanges de haut en bas.

Une tâche devenue inéligible passe à failed sans envoi. Les erreurs temporaires sont retentées après 1, 5, 15 et 30 minutes, puis toutes les 60 minutes jusqu'à expiration. sent signifie que FCM a accepté la requête. L'envoi push est asynchrone et le téléphone ignore les identifiants dupliqués.

Ouvrir le document UML complet

Index des routes API

Au commit dev ad54ffdb2cf359c36cf22b947494195bddcf87fb, le backend déclare 39 opérations HTTP dans les routeurs FastAPI : 36 sous /api/v1 (dont neuf routes marines publiques et deux lectures publiques pour le web) et trois à la racine (/health, /health/ready, /version). Les routes métier et le MVT des signalements requièrent une identité Firebase ; les routes cartographiques marines publiques, /map/web-config, /map/observations et les contrôles de santé sont accessibles sans jeton Firebase. Les routes /api/v1/admin exigent en outre des droits administrateur. Les pages HTML /admin ne sont pas comptées. L'édition et la suppression d'un signalement par son auteur ainsi que GET /users/me/reports restent non implémentées (NW-73, NW-74).

Méthode	Chemin	Objectif
GET	/health	Santé du processus
GET	/health/ready	Disponibilité de la base de données
POST	/users/me	Créer le profil initial
GET	/users/me	Lire le profil
PATCH	/users/me	Modifier le profil/les préférences
DELETE	/users/me	Supprimer le compte
GET	/users/me/boat	Lire le bateau
POST	/users/me/boat	Créer le bateau
PATCH	/users/me/boat	Modifier le bateau
DELETE	/users/me/boat	Supprimer le bateau
PUT	/devices/current	Enregistrer/mettre à jour l'appareil
DELETE	/devices/current	Désactiver l'appareil courant. X-Installation-ID requis.
GET	/map/web-config	Configuration publique Mapbox pour le navigateur ; no-store.
GET	/map/observations	Signalements actifs publics, filtres et pagination par curseur ; no-store.
GET	/map/seamarks/{z}/{x}/{y}.pbf	Marques et balisage ; tuile vectorielle publique.
GET	/map/port-infrastructure/{z}/{x}/{y}.pbf	Équipements portuaires ; tuile vectorielle publique.
GET	/map/harbours/{z}/{x}/{y}.pbf	Ports et lieux portuaires ; tuile vectorielle publique.
GET	/map/depth-contours/{z}/{x}/{y}.pbf	Isobathes ; tuile vectorielle publique.
GET	/map/soundings/{z}/{x}/{y}.pbf	Sondes de profondeur ; tuile vectorielle publique non stockée par cache HTTP.
GET	/map/underwater-hazards/{z}/{x}/{y}.pbf	Dangers immergés ; tuile vectorielle publique.
GET	/map/marine-features/{family}/{z}/{x}/{y}.geojson	Snapshots GeoJSON de famille marine compatible, restrictions de zoom et famille.
GET	/map/marine-sources	Attributions, provenance et usage des sources marines.
GET	/map/bathymetry/{z}/{x}/{y}.png	Bathymétrie visuelle PNG avec cache versionné et masques côtiers.

Méthode	Chemin	Objectif
PUT	/devices/current/position	Remplacer la dernière position. X-Installation-ID requis.
GET	/map/tiles/{z}/{x}/{y}.mvt	Charger une tuile visible de la carte
GET	/reports/{report_id}	Lire un signalement visible
POST	/reports	Créer un signalement manuel/photo
POST	/position-estimates	Calculer une estimation photo
POST	/reports/{report_id}/photo	Envoyer un JPEG
GET	/users/me/alerts	Mon historique d'alertes
GET	/admin/reports	Lister les signalements à modérer
GET	/admin/reports/{report_id}	Lire le détail de modération
PATCH	/admin/reports/{report_id}	Restaurer un signalement supprimé
DELETE	/admin/reports/{report_id}	Supprimer un signalement comme administrateur
PATCH	/admin/reports/{report_id}/photo	Masquer une photo
GET	/admin/users	Lister les comptes
PATCH	/admin/users/{user_id}	Suspendre/réactiver un compte
GET	/admin/moderation-actions	Lire l'audit de modération
GET	/version	Lire la révision Git déployée

Contrat des nouvelles routes marines : toutes les routes listées sous /map/… sont préfixées par /api/v1. Les six familles en .pbf et les snapshots renvoient des données vectorielles en application/vnd.mapbox-vector-tile ou application/geo+json ; /bathymetry renvoie image/png ; /marine-sources renvoie JSON. La plupart des tuiles vectorielles publiques acceptent Z0–Z14 (marques, ports, harbours) ou Z0–Z16 (contours, sondes, dangers) ; la bathymétrie accepte Z0–Z14. Les snapshots GeoJSON acceptent harbours à Z6–Z14, seamarks et underwater-hazards à Z10–Z14, soundings à Z10–Z16 et depth-contours uniquement à Z12 ; port-infrastructure n'est pas accepté. Une combinaison interdite renvoie 422. Les réponses marines peuvent transporter X-NoWave-Marine-Status et X-NoWave-Marine-Sources. Voir backend/app/api/routes/map_tiles.py pour les bornes exactes.

Lectures publiques web (sans Firebase, Cache-Control: no-store) : GET /api/v1/map/web-config renvoie {"mapbox_token": "…"}, uniquement si MAPBOX_PUBLIC_TOKEN respecte le format d'un jeton Mapbox public (pk.) ; sinon la valeur est vide. GET /api/v1/map/observations renvoie {"items": […], "has_more": true, "next_cursor": "…"}, avec has_more=false et sans next_cursor lorsqu'il n'y a plus de page. Le fil mondial peut être restreint par quatre bornes west/south/east/north fournies ensemble (west > east traverse l'antiméridien), category et q (120 caractères maximum). limit vaut 200 par défaut, entre 1 et 500 ; cursor poursuit l'ordre observed_at décroissant puis id décroissant, en conservant les mêmes filtres. Les paramètres invalides renvoient 422.

Le fil public exclut les signalements expirés et retirés et expose seulement id, category, longitude, latitude, observed_at, expires_at, la description non vide et author_username si show_user_name l'autorise. Il n'expose ni photo, ni e-mail, ni identifiant d'auteur, ni informations de bateau ou données administratives. La carte web et la carte d'accueil admin utilisent ce même fil, par pages de 50 observations ; les détails et actions privés restent sous contrôle administrateur. La route statique GEBCO servie par Nginx est distincte de ces 39 opérations FastAPI.

HTTPS est obligatoire. Les données structurées utilisent JSON, les tuiles de signalements et plusieurs tuiles marines utilisent des MVT binaires, les snapshots marins utilisent GeoJSON, la bathymétrie régionale utilise PNG et le fond mondial statique utilise WebP et les photos sont envoyées en multipart. Les modes manuel et photo utilisent

POST /reports.

Initialisation du profil

Firebase Authentication crée ou authentifie d'abord l'identité. Flutter demande ensuite le nom d'utilisateur obligatoire et appelle POST /api/v1/users/me avec le Firebase ID token. Cela crée le profil NoWave ; l'authentification seule ne le crée pas. Le backend vérifie le token, exige email_verified=true et extrait firebase_uid et email de l'identité Firebase vérifiée ; ces champs ne sont jamais acceptés dans le corps de la requête.

Contrôles d'identité et d'autorisation des accès API

Authentification Firebase : le backend vérifie le jeton Firebase transmis par le client. Cette vérification établit l'identité sans imposer nécessairement un profil NoWave actif.

Compte NoWave actif : les routes métier protégées vérifient l'existence et le statut actif du profil. Un compte suspendu est bloqué sur ces routes.

Droits administrateur : les routes d'administration exigent un profil NoWave actif doté du rôle administrateur. Le contrôle est effectué côté backend, indépendamment de l'interface.

Requête de création du profil

```
POST /api/v1/users/me
Authorization: Bearer <Firebase ID token>
Content-Type: application/json

{
  "username": "Jonathan"
}
```

Après création, renvoyer UserProfile (201). Imposer l'unicité du nom d'utilisateur et la limite de 100 caractères. Le serveur contrôle role et status ; show_user_name, show_boat_info et notifications_enabled valent false par défaut. Rejeter un profil ou un nom d'utilisateur en doublon avec 409, une entrée invalide avec 422, un jeton absent ou invalide avec 401 et une identité Firebase dont l'e-mail n'est pas vérifié avec 403.

Opération	Objectif
POST /users/me	Création initiale du profil avec nom d'utilisateur obligatoire.
GET /users/me	Lecture seule : 200 pour un profil existant, 404 avant l'initialisation.
PATCH /users/me	Modifier les champs de profil et de préférences autorisés.
DELETE /users/me	Supprimer/anonymiser les données personnelles NoWave et supprimer l'identité Firebase.

Après la perte de la réponse de création, Flutter peut lire GET /users/me pour déterminer si l'initialisation a réussi. Il ne doit pas attribuer silencieusement un nom d'utilisateur ni s'appuyer sur un GET pour initialiser le profil.

Préférences du compte et règles partagées

Flutter utilise Firebase Authentication pour l'accès au compte. NoWave ne stocke aucun mot de passe. Les routes protégées utilisent l'en-tête Authorization avec le schéma Bearer et un Firebase ID token.

Opération	Entrée et résultat
POST /users/me	Jeton Firebase + username JSON -> UserProfile (201). Créer le profil initial pour l'identité authentifiée ; username est obligatoire et unique. Exiger email_verified=true. firebase_uid et email sont extraits du token Firebase vérifié et ne sont jamais acceptés dans le JSON client. Les préférences de visibilité et de notification valent false par défaut. Les conflits de création renvoient HTTP 409 avec un code distinct : user_already_exists si l'UID Firebase courant possède déjà un profil ; username_already_exists si le nom demandé est déjà utilisé ; email_already_registered si l'adresse e-mail Firebase appartient au profil d'un autre compte. Côté mobile, user_already_exists déclenche une relecture de GET /users/me afin de poursuivre avec le profil existant ; username_already_exists affiche l'erreur de nom déjà utilisé ; email_already_registered n'entraîne aucune relecture du profil du compte courant et affiche un message dédié.
GET /users/me	Sans corps -> UserProfile (200). Lire uniquement un profil existant. Si l'initialisation n'a pas créé le profil, renvoyer 404 ; ce GET n'insère jamais de profil.
PATCH /users/me	JSON partiel -> UserProfile mis à jour (200). Les champs de profil et de préférences autorisés sont indiqués ci-dessous.
DELETE /users/me	Sans corps -> 204 après suppression/anonymisation des données personnelles NoWave et suppression de l'identité Firebase.
Boat GET / POST / PATCH / DELETE	GET -> Boat (200/404), POST -> 201/409 s'il existe déjà, PATCH -> 200, DELETE -> 204. Un bateau optionnel au maximum par utilisateur.

PATCH /users/me - entrée - exemple

```
{
  "username": "Jonathan",
  "show_boat_info": true,
  "notifications_enabled": true
}
```

Exemple de sortie UserProfile

```
{
  "id": "a52c4167-c8d7-4e72-9bd5-90cf39b822c9",
  "username": "Jonathan",
  "date_of_birth": null,
  "nationality": "FR",
  "role": "user",
  "status": "active",
  "show_user_name": false,
  "show_boat_info": true,
  "notifications_enabled": true,
  "created_at": "2026-09-13T09:00:00Z",
  "updated_at": "2026-09-13T10:00:00Z"
}
```

Champs de profil modifiables : username, date_of_birth, nationality, show_user_name, show_boat_info et notifications_enabled. email, firebase_uid, role et status ne sont pas modifiables par cette route. Les réponses UserProfile n'exposent ni firebase_uid ni email. Champs du bateau : name (optionnel), boat_type (obligatoire), flag_country (optionnel). Une mise à jour partielle du bateau ne modifie que les champs fournis.

La suppression du compte retire les données personnelles, les appareils, les jetons, les positions et l'historique personnel des alertes. Les signalements restent avec author_id défini à null ; les liens d'audit vers les comptes ou cibles supprimés deviennent null. Le nettoyage des données NoWave précède la suppression de l'identité Firebase.

Pour rendre ce parcours reprenable côté mobile, l'application enregistre localement un journal temporaire de suppression lié à l'UID Firebase du compte courant avant l'appel DELETE /users/me. Tant que le backend n'a pas confirmé la suppression par HTTP 204, ce journal reste en phase pending, les envois appareil et GPS sont suspendus et la session Firebase locale est conservée afin de permettre une nouvelle tentative. Après le HTTP 204, le journal

passe en phase confirmed ; une reprise éventuelle termine alors uniquement le nettoyage local du jeton FCM, de l'identifiant d'installation et de la session Firebase, sans répéter la suppression backend. Le journal est ignoré s'il ne correspond pas au compte courant et il est supprimé une fois le parcours entièrement terminé.

Enregistrement de l'appareil et position courante

Chaque appareil appartient à un utilisateur authentifié. L'enregistrement utilise un UUID d'installation. Un Device.id conserve son propriétaire. Le Firebase ID token envoyé dans l'en-tête Authorization identifie le compte. Pour PUT /devices/current/position et DELETE /devices/current, l'en-tête X-Installation-ID identifie l'installation mobile et doit correspondre à un appareil appartenant à l'utilisateur authentifié. PUT /devices/current conserve installation_id dans le corps JSON et n'exige pas X-Installation-ID. Lors d'un changement de compte, désactiver l'ancienne installation, supprimer son jeton FCM, sa dernière position et ses tâches de notification pending dans une même transaction coordonnée avec NotificationWorker, effacer l'ancienne session de l'application et enregistrer un nouvel UUID.

PUT /devices/current - entrée - exemple

```
{
  "installation_id": "9d3f6a2e-7237-4dab-9fe9-054b4670d425",
  "platform": "android"
}
```

Opération	Contrat
PUT /devices/current	Entrée JSON avec installation_id, platform et jeton FCM optionnel -> JSON Device (200). La création ou mise à jour est idempotente via installation_id.
Uniqueness	installation_id et chaque fcm_token non nul sont uniques.
DELETE /devices/current	Sans corps, avec X-Installation-ID requis -> 204. La désactivation est idempotente : mettre is_active à false, mettre fcm_token à NULL, supprimer la dernière position et les tâches de notification pending dans une même transaction. Les notifications sent et failed sont conservées.

PUT /devices/current/position - entrée - exemple

```
{
  "position": {"type": "Point", "coordinates": [5.37, 43.29]},
  "accuracy_m": 18.0,
  "heading_deg": 90.0,
  "measured_at": "2026-09-13T10:00:00Z"
}
```

PUT /devices/current/position renvoie JSON DevicePosition (200). Le serveur définit received_at. Conserver une seule dernière position exploitable par appareil. Une mesure plus ancienne que la position enregistrée est rejetée avec 409 et la position valide précédente reste inchangée. Une mesure dont measured_at dépasse de plus de 5 minutes l'heure courante du serveur est rejetée avec 422 et la position valide précédente reste inchangée.

Envoyer une position exploitable au backend au plus une fois toutes les 60 secondes tant que le suivi est actif. Lorsque les alertes de proximité sont activées, le suivi continue en arrière-plan tant que l'application n'est pas fermée de force. La première mesure exploitable après le démarrage du suivi est envoyée immédiatement. Ensuite, une mesure située à moins de 50 m de la dernière position acceptée par le backend n'est pas envoyée, sauf si sa précision est au moins deux fois meilleure. Le flux GPS local n'est pas filtré par cette règle : la carte et la caméra continuent de recevoir les mesures de position. La précision GPS doit être finie et comprise entre 0 et 50 m. Si accuracy_m dépasse 50 m, rejeter la mesure et conserver la position valide précédente. Un échec de validation utilise 422.

Aucune limite d'âge de la position ni aucun filtre de cap ne s'applique à la sélection des destinataires d'alertes.

Déplacer ou zoomer la carte ne met pas à jour DevicePosition et ne déclenche pas de notification.

Créer et lire des signalements (routes implémentées)

Un utilisateur actif peut publier avec POST /api/v1/reports et lire un signalement visible avec GET /api/v1/reports/{report_id}. Les coordonnées GeoJSON sont [longitude, latitude]. Les catégories sont marine_animal, obstruction et pollution. L'édition et le retrait par l'auteur ne sont pas livrés ; les routes administrateur de modération sont distinctes.

POST /reports - entrée manuelle - exemple

```
{
  "client_report_id": "550e8400-e29b-41d4-a716-446655440000",
  "positioning_mode": "manual",
  "category": "pollution",
  "description": "Pollution de surface visible",
  "final_position": {"type": "Point", "coordinates": [5.37, 43.29]},
  "observed_at": "2026-09-13T10:00:00Z"
}
```

Exemple de réponse ReportResponse (HTTP 201 à la création)

```
{
  "id": "f4a15a7c-7c5b-4b92-8b21-53c7e1f81c17",
  "author_id": "8b0a6a27-4c8d-4b76-a3f2-9c7f5e1d6a20",
  "client_report_id": "550e8400-e29b-41d4-a716-446655440000",
  "category": "pollution",
  "positioning_mode": "manual",
  "description": "Pollution de surface visible",
  "final_position": {
    "type": "Point",
    "coordinates": [
      5.37,
      43.29
    ]
  },
  "observed_at": "2026-09-13T10:00:00Z",
  "expires_at": "2026-09-14T10:00:00Z",
  "status": "active",
  "version": 1,
  "created_at": "2026-09-13T10:00:01Z",
  "updated_at": "2026-09-13T10:00:01Z",
  "photo": null
}
```

Opération	Règle et résultat
GET /reports/{id}	200 ReportDetail. La visibilité sur la carte exige status=active et expires_at postérieur à l'heure courante.

Utiliser server_time - 24 h < observed_at <= server_time, expires_at = observed_at + 24 h. Un signalement déjà expiré à sa réception n'est pas publié comme actif et ne crée aucune alerte. Un nouveau client_report_id crée le signalement et renvoie 201 Created. Un client_report_id déjà enregistré pour le même utilisateur avec un contenu identique renvoie le signalement existant avec 200 OK sans créer de doublon ni de nouvelle vague d'alertes. Le même client_report_id réutilisé avec un contenu différent renvoie 409 Conflict.

Le mode manuel ne possède aucune ligne de positionnement ni de photo. La création initiale fixe version à 1. Les opérations de modération et les règles métier correspondantes sont documentées séparément.

Positionnement par photo et envoi

Un signalement photo utilise les champs de création communs ainsi que les mesures de positionnement photo. final_position est le point confirmé ou corrigé par l'utilisateur. FastAPI valide ou recalcule les données de positionnement avant stockage.

Création photo : remplacer le mode et ajouter le positionnement - exemple

```
{
  "positioning_mode": "photo",
  "positioning": {
    "observer_position": {"type": "Point", "coordinates": [5.3779, 43.2945]},
    "gps_accuracy_m": 12.0,
    "azimuth_deg": 145.2,
    "inclination_deg": -3.4,
    "camera_height_m": 2.1,
    "camera_height_source": "device_estimate",
    "camera_height_uncertainty_m": 0.1,
    "focal_length_mm": 24.0,
    "zoom_ratio": 1.0,
    "captured_at": "2026-09-13T09:59:55Z"
  }
}
```

Étape	Échange et résultat
POST /position-estimates	Envoyer la position de capture, la précision GPS, l'azimut, l'inclinaison, la hauteur et les données caméra disponibles. Toutes les mesures correspondent au moment de la capture. Pour le MVP, l'objet est placé au centre du viseur avant le déclenchement. Renvoyer l'estimation (200), sans ID d'estimation ni stockage temporaire.
POST /reports	Envoyer les champs communs avec le positionnement photo. Renvoyer JSON ReportResponse (HTTP 201 à la création, HTTP 200 lors d'une nouvelle tentative identique). Enregistrer ensemble les lignes de positionnement et de photo ; une photo pending ne retarde ni la publication ni les enregistrements d'alerte.
POST /reports/{id}/photo	Envoyer le champ multipart/form-data file contenant le JPEG binaire. FastAPI le valide et l'envoie vers Scaleway ; le téléphone n'utilise pas d'URL d'envoi signée. Le premier succès renvoie 201, le renvoi du même JPEG après une réponse perdue renvoie 200, et un JPEG différent après finalisation renvoie 409. La consultation de la photo passe ensuite par GET /reports/{id}, qui peut fournir une URL de lecture signée de 5 minutes.

Champs de sortie de l'estimation de position

estimated_position est le point GeoJSON estimé ; estimated_distance_m est la distance estimée de l'objet en mètres.

Exemple de sortie d'envoi de photo (201)

```
{
  "report_id": "93c61dc8-f53a-470b-9e83-63020d3f8f62",
  "upload_status": "uploaded",
  "size_bytes": 421350,
  "mime_type": "image/jpeg"
}
```

L'azimut utilise le nord vrai ; l'inclinaison vaut 0 à l'horizon, est positive vers le haut et négative vers le bas. La hauteur est mesurée au-dessus du niveau de la mer. La précision GPS doit être comprise entre 0 et 50 m. Si l'estimation échoue, l'utilisateur définit final_position et le signalement reste en mode photo.

JPEG uniquement, de 1 à 500 000 octets. Flutter corrige l'orientation et supprime les EXIF ; FastAPI valide le JPEG réel, sa taille, son décodage et l'absence d'EXIF. Réessayer un envoi échoué avec le même report_id et la ligne

report_photos existante. Le succès passe upload_status à uploaded ; aucune seconde ligne photo n'est créée. Les objets stockés inutilisés sont supprimés. Le premier succès d'envoi renvoie 201. Un renvoi du même JPEG après une réponse perdue renvoie 200 sans créer de second signalement ni de seconde ligne photo. Un JPEG différent lorsque la photo est déjà finalisée renvoie 409. Une indisponibilité du stockage renvoie 503, conserve le signalement et permet une nouvelle tentative sur la même ligne photo.

ReportDetail nomme l'état d'envoi photo.status et fournit photo.url comme URL de lecture signée temporaire lorsque l'accès est autorisé. PhotoMetadata et report_photos nomment l'état d'envoi upload_status. L'URL signée de lecture est valable 5 minutes. Elle n'est générée que pour une photo uploaded, non masquée, appartenant à un signalement visible. Une URL déjà émise continue jusqu'à son expiration même si la photo est masquée ensuite.

Historiques, accès cartographique et messages push

Les routes d'historique acceptent page et limit. Les réponses paginées contiennent items, page, limit, total et pages. L'exemple ci-dessous présente une alerte.

GET /users/me/alerts - sortie - exemple


```
{
  "items": [
    {
      "id": "a1494250-a731-4ac5-b09b-959cb9906488",
      "report_id": "f4a15a7c-7c5b-4b92-8b21-53c7e1f81c17",
      "report_version": 1,
      "category": "pollution",
      "position": {"type": "Point", "coordinates": [5.37, 43.29]},
      "distance_m": 3334.0,
      "alerted_at": "2026-09-13T10:00:03Z"
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "pages": 1
}
```


GET /api/v1/users/me/alerts renvoie l'historique paginé des alertes de proximité de l'utilisateur. Cet historique est distinct du journal de ses propres signalements, non implémenté dans le MVP connecté. Les entrées d'alerte restent un instantané par utilisateur, signalement et version même si FCM échoue ; elles n'ont pas d'état lu/non lu.

Contenu du push	Objectif
Catégorie, position DMS, distance	Informations affichées au destinataire.
Identifiants stables et version du signalement	Associer l'alerte à sa notification, au signalement et à sa version ; ignorer les ID d'alerte dupliqués.
Expiration	Définir l'expiration du push sur l'expiration du signalement.

Lors de la création ou de la modification d'un signalement, sélectionner les autres utilisateurs actifs ayant donné leur consentement aux notifications et disposant d'appareils actifs, d'un jeton FCM et d'un GPS exploitable à moins de 18 520 m. Aucun filtre de cap ou d'ancienneté de position ne s'applique. Créer une alerte par utilisateur/signalement/version et une tâche par appareil éligible/signalement/version. La distance d'historique correspond au minimum parmi les appareils éligibles de l'utilisateur.

Un worker traite les tâches validées et conserve le même UUID de tâche lors des nouvelles tentatives. Avant chaque envoi, il revérifie la version et la visibilité du signalement, son expiration, le statut et le consentement du destinataire, ainsi que le propriétaire, l'activité et le jeton de l'appareil. La distance reste celle de la sélection initiale et n'est pas recalculée lors de l'envoi. sent signifie que FCM a accepté la requête. Les échecs temporaires

sont retentés après 1, 5, 15 et 30 minutes, puis toutes les 60 minutes ; ils respectent un Retry-After plus long et s'arrêtent avant l'expiration. Conserver updated_at terminal inchangé, supprimer les tâches sent/failed sept jours après cette dernière modification. L'historique est séparé.

GET /map/tiles/{z}/{x}/{y}.mvt renvoie application/vnd.mapbox-vector-tile avec les paramètres optionnels category et report_id. Une feature de signalement contient report_id, category et une géométrie Point. Un cluster peut contenir cluster=true, cluster_count et une géométrie. Seuls les signalements actifs et non expirés apparaissent. Les descriptions, photos et auteurs sont chargés séparément. La consultation est mondiale. Les 10 NM servent uniquement à sélectionner les destinataires des notifications.

Administration et audit

Chaque opération /admin exige un administrateur authentifié et actif. Un utilisateur authentifié sans rôle admin est refusé ; un administrateur suspendu est refusé avant l'accès aux données administratives. Chacune des cinq actions de modération exige un motif contenant du texte autre que des espaces. Rejeter un motif absent, vide ou composé uniquement d'espaces avec 422. Enregistrer la modification d'état et l'entrée d'audit immuable dans une même transaction.

Accès web à l'administration

L'interface d'administration du MVP est rendue côté serveur par FastAPI et Jinja et est publiée sous /admin. Le formulaire de connexion envoie l'e-mail et le mot de passe à Firebase Authentication ; NoWave ne stocke jamais le mot de passe. Après authentification, FastAPI exige une identité dont l'e-mail est vérifié, un profil NoWave actif et le rôle admin avant de créer la session web.

La session d'administration est conservée dans le cookie __Host-nowave_admin_session, valable 8 heures, avec HttpOnly, Secure, SameSite=Strict et Path=/. À chaque requête administrative, FastAPI revérifie la session Firebase, y compris sa révocation, puis relit le profil NoWave afin qu'une suspension ou un changement de rôle s'applique dès la requête suivante. Ce contrôle de révocation systématique concerne la session web d'administration ; le contrat mobile continue d'utiliser le Firebase ID token selon les règles décrites dans la section Identité et accès.

Chaque requête POST de l'interface web est protégée contre les attaques CSRF. La connexion utilise un cookie CSRF distinct __Host-nowave_admin_csrf en double soumission ; les formulaires d'une session ouverte utilisent un jeton dérivé de la session. La déconnexion efface les cookies du navigateur mais ne révoque pas les jetons Firebase de l'application mobile. Les pages administratives utilisent Cache-Control: no-store, l'échappement automatique Jinja et ne sont pas exposées dans OpenAPI.

La connexion web nécessite la clé Firebase Web API. En production, elle doit être fournie au conteneur FastAPI par FIREBASE_WEB_API_KEY_FILE pointant vers /run/secrets/firebase_web_api_key. Le secret source est conservé hors du dépôt Git, dans /opt/nowave/secrets/firebase_web_api_key sur le VPS. Si cette configuration est absente, l'API reste démarrée mais POST /admin/login renvoie HTTP 503.

Pagination des lectures administrateur

Les listes JSON sous /api/v1/admin utilisent le format commun {"items": [...], "page": 1, "limit": 20, "total": 0, "pages": 0}. page vaut 1 par défaut. limit vaut 20 par défaut et ne peut pas dépasser 100. Une valeur de pagination hors bornes renvoie HTTP 422. Une page valide située au-delà de la dernière page renvoie items vide tout en conservant total et pages.

Opération	Action autorisée / sortie
GET /admin/reports	Paramètres optionnels : page, limit et status=active|expired|removed. Renvoie 200 avec la liste paginée des signalements du plus récent au plus ancien. Le statut exposé est le statut effectif : une ligne stockée active dont expires_at est dépassé est exposée comme expired ; removed reste removed. Chaque élément expose id, category, positioning_mode, status, observed_at, expires_at, removed_at, version, author_id, author_username, photo_status et photo_hidden_at. Les signalements retirés et expirés restent consultables par l'administration.
GET /admin/reports/{id}	report_id -> 200 avec le détail destiné à la modération. La réponse reprend ReportDetail et ajoute removed_at, un auteur complet (id, username, email, status, deleted) et photo.hidden_at. La vue administrateur reste accessible pour un signalement active, expired ou removed et ignore les préférences show_user_name et show_boat_info. Si l'auteur a été supprimé, ses données personnelles sont absentes et author.deleted vaut true. Un identifiant inexistant renvoie 404.
PATCH /admin/reports/{id}	JSON status=active et reason -> ReportDetail (200). Restaurer à active avant l'expiration ; après expiration, le signalement reste expired. Conserver observed_at et expires_at.
DELETE /admin/reports/{id}	Corps JSON avec reason -> 204. Supprimer logiquement le signalement et enregistrer son entrée d'audit dans la même transaction.
PATCH /admin/reports/{id}/photo	JSON hidden=true et reason -> PhotoMetadata (200). Masquer la photo, conserver son état d'envoi et le signalement.
GET /admin/users	Paramètres optionnels : page, limit et status=active|suspended. Renvoie 200 avec la liste paginée des comptes. Chaque élément expose id, username, email, role, status, created_at, updated_at et reports_count. reports_count correspond au nombre de signalements encore associés au compte. Le filtrage par status est effectué avant la pagination.
PATCH /admin/users/{id}	JSON status=suspended ou active et reason -> UserProfile (200). Auditer séparément la suspension et la réactivation.
GET /admin/moderation-actions	page, limit -> 200 avec le journal d'audit paginé, de l'action la plus récente à la plus ancienne. Chaque entrée peut exposer id, action, admin_user_id, admin_username, target_user_id, target_report_id, target_photo_report_id, reason, previous_status, new_status et created_at. La ressource est en lecture seule : POST, PUT, PATCH et DELETE ne sont pas fournis et renvoient 405. Lorsqu'un administrateur ou une cible a été supprimé avec SET NULL, l'entrée d'audit reste disponible avec l'identifiant correspondant à null.

Les pages /admin/users et /admin/reports proposent search (80 caractères), statut et pagination. /admin/users/{user_id}/reports liste 20 signalements par page avec has_more. Les lectures /admin/data/... restent protégées. Le détail HTML filtre l'audit par signalement et utilise history_page.

Détail administrateur et visibilité

Le détail mobile authentifié GET /reports/{id} reste limité aux signalements actifs et non expirés. Le détail administrateur GET /admin/reports/{id} peut lire les signalements actifs, expirés et retirés pour les besoins de modération.

Photos dans les lectures administrateur

Dans le détail administrateur, une URL de lecture signée temporaire peut être générée pour une photo uploaded associée à un signalement actif, expiré ou retiré. L'interface HTML affiche la photo autorisée et un lien pour l'ouvrir avant modération. Une photo dont hidden_at est renseigné ne reçoit jamais de nouvelle URL de lecture, y compris pour un administrateur.

Journal d’audit en lecture seule

Le journal d’audit reste disponible après suppression d’un compte ou d’une cible lorsque la clé étrangère correspondante est mise à null. Les lectures de ce journal ne modifient aucun état métier.

Exemples de requêtes de modération

Tous les exemples utilisent Content-Type: application/json et un administrateur actif authentifié. Le motif doit rester non vide après suppression des espaces en début et fin.

Restaurer un signalement avec PATCH /admin/reports/{id}

```
{
  "status": "active",
  "reason": "Signalement vérifié après examen"
}
```

Masquer une photo avec PATCH /admin/reports/{id}/photo

```
{
  "hidden": true,
  "reason": "La photo enfreint les règles de modération"
}
```

Suspendre un utilisateur avec PATCH /admin/users/{id}

```
{
  "status": "suspended",
  "reason": "Signalements abusifs répétés"
}
```

Supprimer un signalement avec DELETE /admin/reports/{id}

```
{
  "reason": "Faux signalement confirmé lors de l’examen"
}
```

Réactiver un utilisateur avec PATCH /admin/users/{id}

```
{
  "status": "active",
  "reason": "Compte examiné et suspension levée"
}
```

La route DELETE administrateur accepte un corps de requête JSON contenant reason et renvoie 204 sans corps de réponse. Les tests client et API doivent conserver ce corps. Il s’agit d’une suppression logique du signalement, et non d’une suppression physique.

L’audit enregistre l’action, l’administrateur, la cible, le motif, le statut précédent/nouveau lorsque cela s’applique, ainsi que la date de création. Une nouvelle entrée possède une cible correspondant à son action. Les entrées d’audit ne peuvent pas être modifiées ; GET /admin/moderation-actions est une lecture seule et les méthodes POST, PUT, PATCH et DELETE ne sont pas fournies sur cette ressource. La suppression d’un compte ou d’une cible peut effacer le lien correspondant avec SET NULL sans supprimer l’entrée d’audit.

La suspension bloque la publication et la réception d’alertes ; les signalements existants restent présents. Masquer une photo conserve son état d’envoi et bloque la création de nouvelles URL de lecture, y compris dans les lectures administrateur. Une restauration conserve l’expiration d’origine. La modification de modération et l’entrée d’audit sont enregistrées dans une même transaction.

Services externes et intégrations

Les services externes fournissent l'identité, le transport des messages, le stockage objet, le fond Mapbox Standard, les données marines complémentaires et l'hébergement. L'autorisation métier, la sélection des destinataires et la durée de vie des signalements restent contrôlées par FastAPI et PostgreSQL/PostGIS.

Service ou SDK	Utilisé par	Objectif
Firebase Authentication	Flutter + FastAPI	Identité et jetons, authentification e-mail/mot de passe pour le MVP.
Firebase Cloud Messaging	worker FastAPI -> mobile	Transporter les notifications push vers les installations Android et iOS éligibles.
Scaleway Object Storage	FastAPI	Stocker les JPEG validés dans un bucket privé, fournir des URL de lecture signées de 5 minutes lorsqu'elles sont autorisées et permettre le nettoyage serveur des objets non référencés.
Fournisseurs publics de données marines	FastAPI / adaptateurs marins	Contribuer aux couches de bathymétrie, sondes, isobathes, marques et équipements portuaires après normalisation, fusion et attribution des sources par NoWave. Le fond rendu reste Mapbox Standard ; ces données ne constituent pas une carte de navigation officielle.
Mapbox SDK	Flutter et interfaces web	Rendre le globe et le fond Mapbox Standard ainsi que les couches marines et les signalements NoWave dans l'application Flutter (Mapbox Maps Flutter) et sur les cartes web (Mapbox GL JS).
Hostinger VPS + Docker + Cloudflare	Backend / base de données / exposition publique	Héberger FastAPI et PostgreSQL/PostGIS dans des conteneurs Docker sur un VPS Hostinger. Conserver PostgreSQL privé et utiliser Cloudflare pour protéger les services publics.

La couche MVT des signalements NoWave est produite par l'API depuis PostGIS ; elle ne doit pas être confondue avec les tuiles marines publiques issues des adaptateurs fournisseurs. Firebase Cloud Messaging ne décide pas quels utilisateurs doivent recevoir un signalement : NotificationService et PostGIS s'en chargent.

Configuration Scaleway

Le backend utilise PHOTO_STORAGE_BUCKET et PHOTO_STORAGE_REGION, avec fr-par comme région par défaut. Les identifiants SCW_ACCESS_KEY et SCW_SECRET_KEY restent hors de Git et peuvent être fournis en production par SCW_ACCESS_KEY_FILE et SCW_SECRET_KEY_FILE. Les appels de stockage utilisent des délais de connexion et de lecture courts sans retry automatique du SDK ; les nouvelles tentatives fonctionnelles restent pilotées par NoWave.

Formats API, confidentialité et erreurs

Les données structurées de requête et de réponse utilisent JSON. Les coordonnées utilisent GeoJSON [longitude, latitude]. L'envoi de photo utilise multipart/form-data ; les tuiles cartographiques des signalements et certaines tuiles marines utilisent des MVT binaires ; les snapshots marins utilisent GeoJSON et la bathymétrie régionale des images PNG et le fond mondial statique des images WebP. Les dates utilisent des valeurs date et les horodatages utilisent UTC.

Champ / famille	Validation
Profil / bateau	Nom d'utilisateur unique jusqu'à 100 caractères, nom de bateau optionnel jusqu'à 100 caractères, codes pays à deux lettres.
Valeurs bateau	voilier, bateau_moteur, catamaran, semi_rigide, jet_ski, autre.
Signalement	Description jusqu'à 250 caractères. Catégories : marine_animal, obstruction, pollution. Modes : manual, photo. Heure d'observation strictement moins de 24 heures avant l'heure serveur et jamais dans le futur.
Coordonnées / capteurs	Point WGS-84, longitude puis latitude. Précision GPS finie, 0-50 m, cap et azimut de 0 inclus à 360 exclus, hauteur de caméra positive.
Photo	JPEG, 1-500 000 octets, avec EXIF supprimé. Le mode manuel n'a pas de photo ; les estimations dérivées d'une photo peuvent être absentes.

Limites techniques de l'envoi photo

La requête multipart de POST /reports/{report_id}/photo est limitée à 512 000 octets avant parsing afin de laisser la marge d'enveloppe nécessaire au JPEG de 500 000 octets. Nginx conserve une limite de 32 Ko sur les autres routes.

Exemple JSON d'erreur courante

```
{
  "error": {
    "code": "gps_precision_insufficient",
    "message": "La précision GPS doit être de 50 mètres ou meilleure.",
    "details": {"accuracy_m": 72, "maximum_accuracy_m": 50}
  }
}
```

HTTP	Signification / famille concernée
200 / 201 / 204	Lecture ou mise à jour réussie / ressource créée / suppression ou désactivation réussie sans corps de réponse.
400 / 422	Syntaxe invalide ou paramètre incohérent / échec de validation d'un champ ou d'une règle métier.
401 / 403	Jeton Firebase absent, invalide ou expiré / utilisateur authentifié sans permission, sans rôle admin ou dont le compte NoWave est suspended. Un compte suspendu reçoit HTTP 403 avec le code user_inactive sur les routes métier. Pour POST /users/me, 403 s'applique aussi lorsque l'e-mail Firebase n'est pas vérifié.
404	Les ressources n'existent pas. La recherche du bateau renvoie 404 lorsqu'aucun bateau n'est enregistré.
409	Conflit d'état. Pour POST /users/me, distinguer user_already_exists (l'UID Firebase courant possède déjà un profil), username_already_exists (nom d'utilisateur déjà pris) et email_already_registered (l'adresse e-mail appartient au profil d'un autre compte). Les autres cas incluent notamment un second bateau, la suppression d'un signalement incompatible avec son état ou l'envoi d'un JPEG différent après finalisation d'une photo.
503	Une dépendance requise, comme PostgreSQL/PostGIS, Scaleway Object Storage ou un fournisseur marin, est indisponible. Un échec de stockage photo conserve le signalement et autorise une nouvelle tentative.
403	admin_required. droits administrateur insuffisants ; photo_upload_forbidden. envoi photo non autorisé.

HTTP	Signification / famille concernée
409	report_client_id_conflict. identifiant réutilisé avec un contenu différent ; photo_already_uploaded. photo différente après finalisation ; device_position_stale. position plus ancienne que la dernière acceptée.
422	device_position_future. date de mesure trop avancée ; gps_precision_insufficient. précision GPS supérieure à 50 m.
503	photo_storage_unavailable. service de stockage des photos indisponible.

Les noms d'utilisateur visibles exigent show_user_name ; le nom et le type du bateau exigent show_boat_info. La date de naissance et la nationalité restent privées. Les signalements conservés après suppression d'un compte affichent « Utilisateur supprimé ». Seules les photos uploaded, non masquées, de signalements visibles reçoivent de nouvelles URL de lecture signées ; les URL existantes expirent séparément.

Les contrôles de santé sont publics. GET /health renvoie {"status":"ok"} (200). GET /health/ready renvoie {"status":"ready","database":"ok"} (200), ou {"status":"not_ready","database":"unavailable"} (503). L'objet d'erreur commun utilise un code stable en lower_snake_case, un message lisible et des détails optionnels.

Exigences non fonctionnelles

Ces exigences regroupent les contraintes transversales utilisées par l'implémentation et la QA. Elles complètent les user stories fonctionnelles et les contrats API.

Domaine	Objectif NoWave
Sécurité	Authentification Firebase, autorisation backend par rôle, statut et propriété, aucun mot de passe stocké, aucun secret dans le dépôt.
Confidentialité	Visibilité du profil sur consentement, EXIF supprimé, uniquement la dernière position exploitable de l'appareil, sans historique de localisation.
Résilience	Reprise des requêtes idempotentes après interruption réseau. Le JPEG peut être conservé temporairement pendant le parcours de publication ouvert afin de permettre un nouvel envoi avec la même ligne photo. Aucune file persistante locale ni reprise automatique du JPEG après fermeture du parcours n'est prévue. Le stockage photo utilise un worker serveur de nettoyage des objets non référencés ; la file persistante de notifications FCM reste gérée côté serveur avec des nouvelles tentatives tenant compte de l'expiration.
Performance	Index spatiaux GiST, chargement MVT de la zone visible, envois JPEG compacts limités à 500 000 octets.
Maintenabilité	Monolithe modulaire avec architecture hexagonale, frontières de domaine explicites et ports/adaptateurs.
Testabilité	Tests unitaires, API, intégration, PostGIS, Flutter, sécurité et terrain sélectionnés selon la modification.
Traçabilité	Signalements versionnés, audit de modération immuable, issue Linear -> pull request -> preuve QA enregistrée.

Contrôles d'acceptation des règles corrigées

Vérifier la création du profil via POST et sa lecture par GET, l'exclusion des signalements par les tuiles à expires_at sans job d'expiration et le cache HTTP MVT de 15 secondes. Ne pas imposer une disparition visuelle à la seconde près sur Flutter ; ne pas tester comme exigence MVP le journal des signalements personnels hors engagement (NW-74). Tester les motifs pour les cinq actions de modération, gps_precision_insufficient au-dessus de 50 m, et une seule ligne photo après nouvelles tentatives. Aucun TTL d'ancienneté GPS ne filtre les destinataires.

NW-163. Acceptation du parcours FCM vers la carte : vérifier l'ouverture directe sur la même carte pour l'action « Ouvrir » au premier plan, un tap depuis l'arrière-plan et getInitialMessage après démarrage à froid ; aucun écran de rapport préalable ni empilement. Vérifier le marqueur sélectionné visible indépendamment des MVT, le cadrage dynamique sur signalement + position locale avec padding et plusieurs distances, la flèche avec cap disponible ou le repère neutre sans cap, la liberté de manipulation de la carte et l'absence de recentrage parasite par le suivi. Couvrir les positions absentes ou arrivant en différé, la carte pas encore initialisée, le refus de localisation, le réseau indisponible, les signalements expirés/retirés/404, le formulaire de publication en cours, les doublons FCM, plusieurs alertes distinctes, la reprise explicite après échec, la déconnexion/changement de compte et l'ouverture volontaire de la fiche depuis l'historique ou un marqueur. Noter séparément les résultats des tests simulés et des essais FCM/APNs sur téléphones Android/iOS réels ; une simulation ne prouve pas une livraison sur appareil.

Contrôles de recette cartographique complémentaires (révision 28/10/2026) : pour les routes marines publiques, contrôler les codes 200/422/429/503, le statut X-NoWave-Marine-Status, les en-têtes Cache-Control, la présence d'attribution et les bornes de zoom. Tester le maintien d'une ancienne géométrie après une réponse partielle vide, la relève progressive de Z6 à Z14 sous budget et le report des marges avant le détail visible. Vérifier le fond mondial, l'emprise EMODnet et les pannes raster 503 sans remplacement par un PNG vide. Sur Android et iOS réels, contrôler la persistance des isobathes et sondes pendant les zooms, la différenciation des marques, les bords côtiers et la fluidité de la carte avec des zones denses. Distinguer succès des tests hôtes, disponibilité des fournisseurs et recette de rendu sur appareil.

Contrôles qualité

Choisir les tests en fonction de la modification ; chaque PR n'a pas besoin de tous les niveaux de test. Enregistrer la configuration, l'action, le résultat attendu, le résultat observé et les preuves dans l'issue Linear ou la pull request.

Scénario	Résultat attendu
Compte et accès	Créer le compte Firebase, puis POST /users/me avec username. GET ne fait que lire. Vérifier l'unicité du nom d'utilisateur, l'absence de profil, les jetons, rôles et la propriété. Vérifier les trois conflits 409 de création : user_already_exists doit déclencher une relecture de GET /users/me et poursuivre avec le profil existant ; username_already_exists doit signaler le nom déjà utilisé ; email_already_registered doit afficher un message dédié sans relecture du profil du compte courant. Vérifier qu'un compte suspended peut encore lire GET /users/me et nettoyer l'appareil avec DELETE /devices/current, mais qu'il reçoit user_inactive sur les routes métier et n'accède plus à HomeScreen.
Carte et signalements	Parcourir les tuiles, ouvrir les détails courants et publier des signalements manuels et photo. Vérifier la confirmation de final_position et l'expiration d'origine. Les signalements removed et expired restent hors de la vue cartographique authentifiée.
Interruption réseau et stockage	Réessayer client_report_id après une réponse perdue ne crée qu'un seul signalement et une seule vague d'alertes. Un retry photo échoué réutilise le même report_id et la même ligne photo, puis passe à uploaded après un envoi réussi. Vérifier également qu'une fermeture du parcours avant la réussite de l'envoi ne crée aucune reprise automatique lors d'une ouverture ultérieure. Le signalement publié reste conservé et la photo reste indisponible.
Limites numériques	Tester le GPS à 50 m et au-delà, l'erreur gps_precision_insufficient, la limite d'alerte à 18 520 m, la borne stricte d'observation avec une valeur juste avant 24 heures acceptée et une valeur exactement à 24 heures refusée, l'expiration dynamique à 24 heures, le texte à 250 caractères et le JPEG à 500 000 octets et au-delà.
Alertes de proximité	Un utilisateur éligible avec deux appareils produit deux tâches et une entrée d'historique par version de signalement. Revérifier le consentement, la qualité GPS, la version du signalement et l'éligibilité du destinataire. L'historique subsiste en cas d'échec du push.

Scénario	Résultat attendu
Récupération après panne	Simuler des coupures réseau/API, des pannes de stockage et des erreurs FCM temporaires. Vérifier les retries et le comportement dégradé. Pour le stockage photo, vérifier qu'un retry identique reste idempotent, qu'un objet non référencé âgé d'au moins 1 heure est supprimé par photo_cleanup et qu'une photo référencée est conservée. Redémarrer les workers et vérifier que le nettoyage rescane le bucket et que le téléphone ignore les identifiants d'alerte dupliqués.
Confidentialité et modération	Vérifier la confidentialité et la propriété. Les cinq actions de modération rejettent un motif absent, vide ou composé uniquement d'espaces. Chaque action réussie et son audit sont validés ensemble.

Couverture des tests et contrôles terrain

Les tests unitaires couvrent les règles métier, la validation et les calculs. Les tests API vérifient les routes FastAPI, l'authentification Firebase, les permissions, les codes de statut et les erreurs. Les tests d'intégration couvrent PostgreSQL/PostGIS et les adaptateurs externes. Les contrôles de base de données couvrent les contraintes, les migrations, les index géographiques et les recherches ST_DWithin dans un rayon de 10 NM. Les tests Flutter vérifient les widgets, les formulaires, les changements d'état et les principaux parcours utilisateur. Les tests web couvrent le composant cartographique partagé ; les parcours Playwright administrateur utilisent PostgreSQL/PostGIS isolé, avec identité et stockage simulés. Les contrôles de sécurité couvrent les jetons, les rôles, la propriété, les comptes suspendus et la validation JPEG.

À quai et en mer, vérifier les seuils GPS et les entrées des capteurs. Comparer les positions estimées et confirmées avec des cibles connues ; enregistrer les téléphones, les conditions, le nombre d'essais et les échecs.

Preuves avant merge

Les tests automatisés doivent être reproductibles depuis le dépôt et réussir localement et/ou en CI. Vérifier que les fonctionnalités existantes fonctionnent toujours et ajouter si possible un test de régression pour les corrections de bugs. Enregistrer les résultats dans la PR. Valider manuellement l'UI, les permissions, les capteurs et les interactions avec les fournisseurs lorsque c'est pertinent. Un échec de test critique bloque le merge. Pour Scaleway Object Storage, les doublures de test ne remplacent pas une vérification sur le fournisseur réel : valider au minimum un upload dans le bucket privé, une lecture par URL signée et le nettoyage d'un objet non référencé.

Gestion du code source et définition de terminé

Linear suit le travail, GitHub gère le code et les revues, et Discord sert à la communication de l'équipe. Jonathan, Vadim et Brice suivent les mêmes règles, y compris le propriétaire du dépôt.

Branche	Utilisation
main	Référence des releases stables après QA et PR de dev vers main. Aucun déploiement autonome sur push main n'est configuré dans le workflow actuel.
dev	Intégration partagée : un push sur dev validé par tous les contrôles CI déclenche le déploiement du commit exact sur le VPS actuel. Un tel déploiement ne vaut pas release stable.
Branche de travail	Créée depuis dev pour une modification distincte liée si possible à une issue Linear.

Aucun développement direct ni push direct vers dev ou main. Aucune branche de travail ne fusionne directement dans main. Utiliser feat/, fix/, docs/, refactor/, test/ ou chore/, suivis de l'ID Linear et d'un nom court.

De l’issue à la release

CON-026 (option A). Les PR vers dev et main déclenchent des validations seulement ; un push dev dont les contrôles réussissent appelle production.yml et rend éligible au déploiement la révision exacte sur le VPS Hostinger, sous réserve des protections de l’environnement production et des prérequis d’accès SSH. La promotion de dev vers main constitue une décision de release après QA, mais ne déclenche actuellement aucun déploiement autonome supplémentaire. Voir deploy/README.md. Le VPS en ligne ne doit pas être appelé « staging » sans vérification de sa fonction réelle.

1. Sélectionner ou s’attribuer une issue Linear avec un objectif clair, le travail à réaliser et les critères d’acceptation, puis la déplacer vers le statut de travail actif.
2. Mettre dev à jour, créer la branche de travail et réaliser de petits commits clairs. Des préfixes simples comme feat, fix ou docs peuvent être utilisés.
3. Ouvrir une PR vers dev intitulée [<LINEAR-ID>] courte description. Inclure l’ID de l’issue et son lien direct ; sans eux, la PR n’est pas prête pour la revue.
4. Décrire ce qui a changé et pourquoi, les critères terminés, les tests et leurs résultats. Ajouter les preuves utiles ainsi que les limites pertinentes, migrations ou travaux de suivi.
5. Obtenir l’approbation d’au moins un autre membre de l’équipe. Résoudre les commentaires bloquants et les échecs de tests critiques, puis merger vers dev et valider.
6. Pour une release, terminer la QA, tester les principales fonctionnalités du MVP et vérifier le fonctionnement après déploiement. Faire passer la version stable via une PR de dev vers main.

Contrôles et responsabilités

Limiter les modifications au périmètre de l’issue. Examiner les migrations et les contraintes lorsque le schéma change. Ne jamais committer ni journaliser de secrets, jetons, mots de passe ou données sensibles. Mettre la documentation à jour lorsque le comportement, l’API, l’architecture ou la configuration changent.

L’auteur implémente, teste et explique la modification. Le reviewer vérifie le code, le périmètre, les tests, la sécurité et la cohérence. L’équipe maintient dev fonctionnelle et décide quand elle est prête pour main.

Traçabilité de la révision documentaire (28/10/2026) : sources vérifiées sur FBCIFY/NoWave@ad54ffdb2cf359c36cf22b947494195bddcf87fb : backend/app/api/routes/map_tiles.py, backend/app/api/router.py, map_web_config.py, public_observations.py, backend/app/web/routes/admin_pages.py, backend/app/infrastructure/repositories/postgresql_public_observation_repository.py, web/src/map-view.js, backend/app/infrastructure/marine/, mobile/lib/core/map/, mobile/lib/features/map/data/, mobile/lib/features/profile/, mobile/lib/core/api/api_config.dart, mobile/lib/core/map/marine/marine_zoom.dart, compose.yaml, deploy/compose.yaml, deploy/local/, deploy/web/, deploy/marine/README.md, .github/workflows/checks.yml et production.yml. Les schémas UML/MCD de la version initiale décrivent les entités métier historiques ; les extensions de cartographie marine sont explicitées dans ce document sans modifier artificiellement le modèle SQL. Les changements ultérieurs sur dev exigent une nouvelle vérification.

Définition de terminé

Marquer l’issue Linear Done uniquement après réalisation complète de ses critères d’acceptation, réussite des contrôles automatisés et manuels pertinents, approbation par un autre membre, résolution des commentaires bloquants et mise à jour de la documentation. La modification doit être mergée dans dev et validée.

Annexe des schémas techniques

Cette annexe rassemble les six diagrammes détaillés du modèle métier et des échanges. Les diagrammes complets restent présents dans leurs chapitres d'origine. Les planches suivantes en proposent une lecture continue avec des dimensions et des espacements harmonisés. Le modèle conceptuel de données est également repris en trois vues détaillées.

Index des diagrammes

A01 Comptes appareils et signalements

Chapitre d'origine UML comptes, appareils et signalements Page 23

A02 Signalements photos et alertes

Chapitre d'origine UML signalements photo et alertes Page 24

A03 Modération et audit

Chapitre d'origine UML modération et audit Page 25

A04 Positionnement et création

Chapitre d'origine Séquence 1 publier un signalement photo Page 38

A05 Cartographie et consultation

Chapitre d'origine Séquence 2 consulter la carte Page 39

A06 Alertes et notifications

Chapitre d'origine Séquence 3 alerter les utilisateurs à proximité Page 40

M01 MCD cœur métier

Chapitre d'origine Architecture du système puis MCD Page 12

M02 MCD notifications et historique

Chapitre d'origine Architecture du système puis MCD Page 12

M03 MCD modération

Chapitre d'origine Architecture du système puis MCD Page 12

Les diagrammes UML sont présentés en portrait et les séquences en paysage. Dans les séquences, seuls les intervalles vides des lignes de vie sont raccourcis. Les participants, les messages, leur ordre, les flèches et les annotations sont conservés.

A01 Comptes appareils et signalements

Chapitre d'origine UML comptes, appareils et signalements Page 23

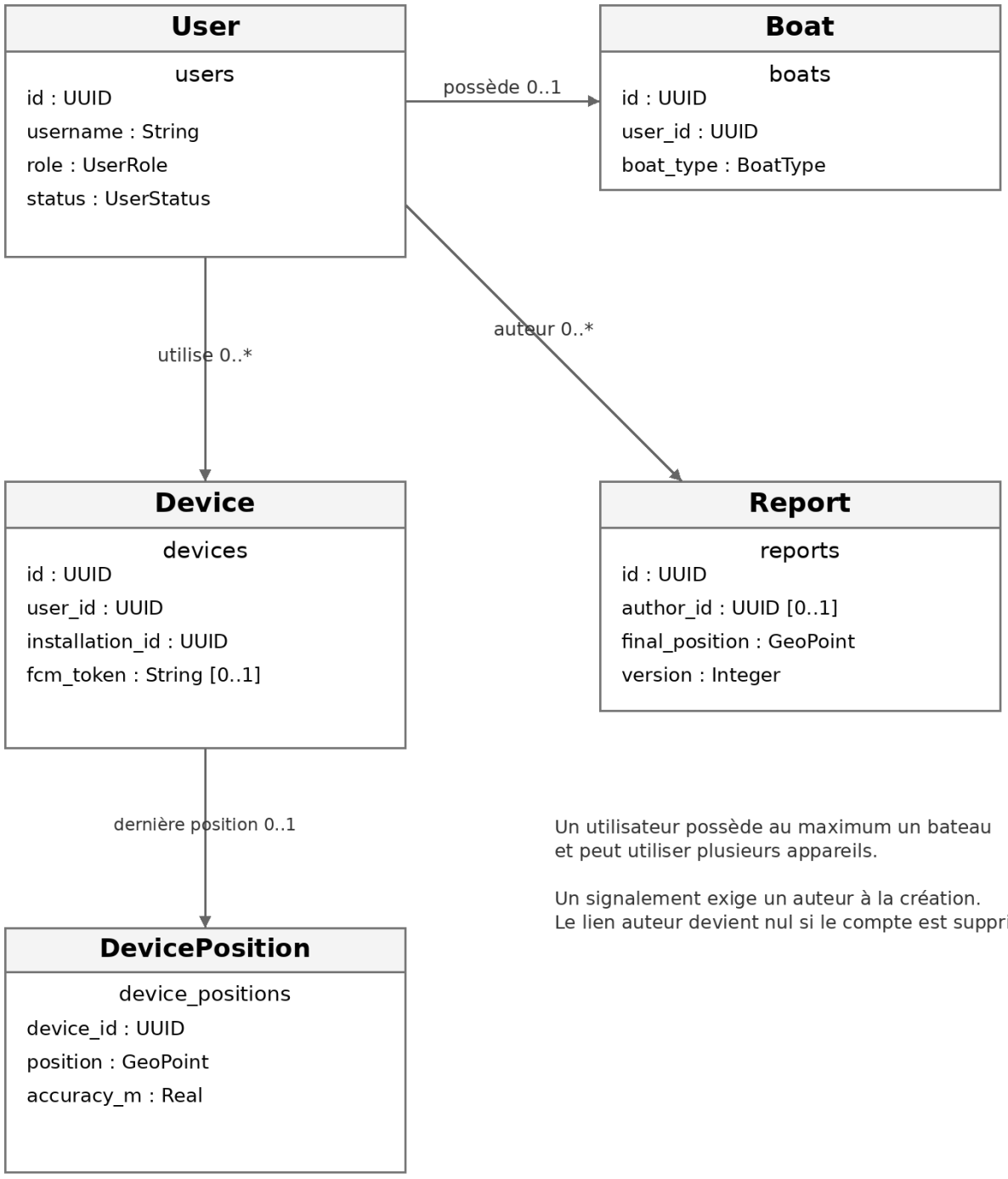
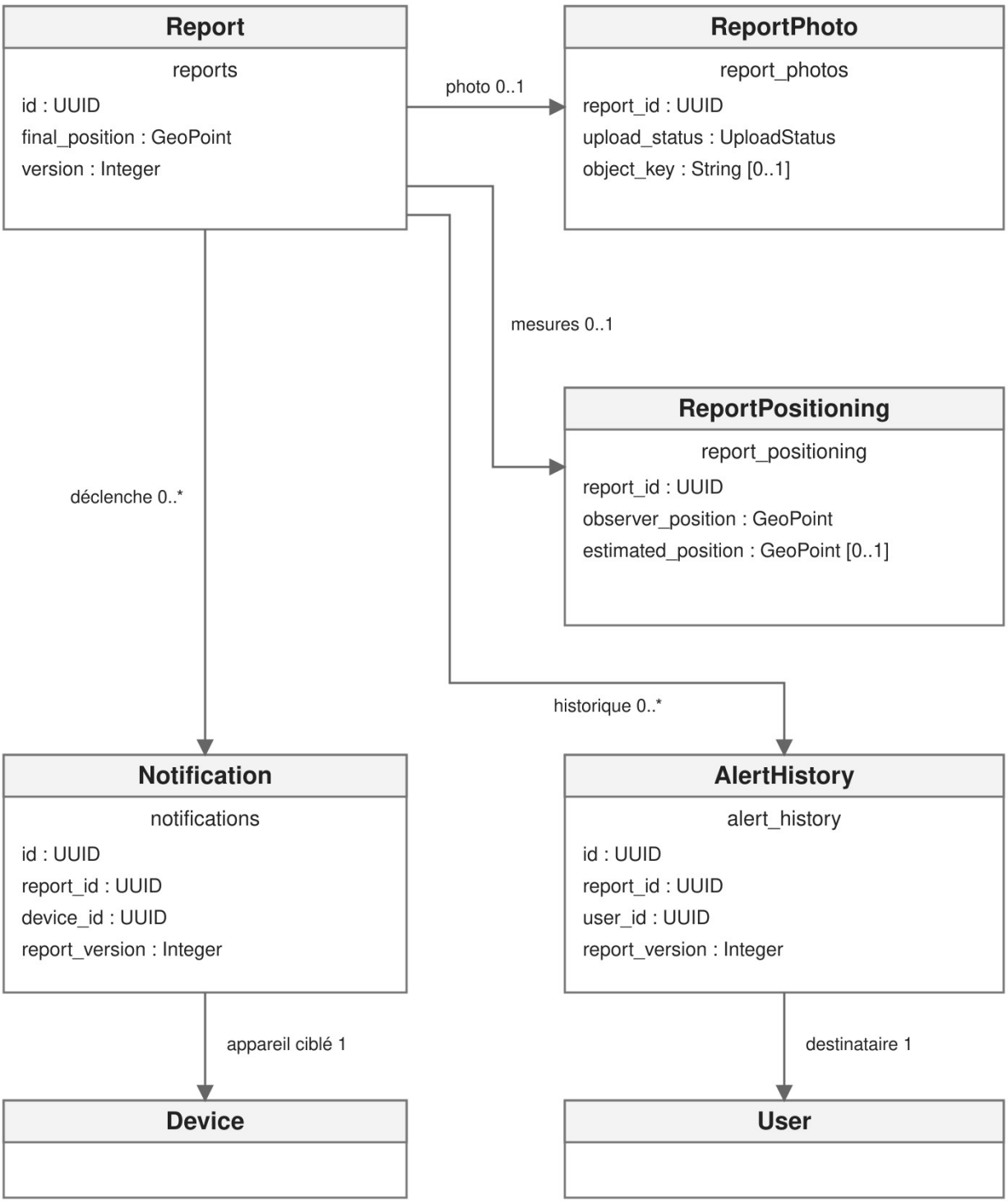


Planche A01 Diagramme complet

A02 Signalements photos et alertes

Chapitre d'origine UML signalements photo et alertes Page 24



Le mode photo possède une ligne de positionnement et une ligne photo ; le mode manuel n'en possède aucune. L'historique est par utilisateur, les tâches d'envoi sont par appareil.

Planche A02 Diagramme complet

A03 Modération et audit

Chapitre d'origine UML modération et audit Page 25

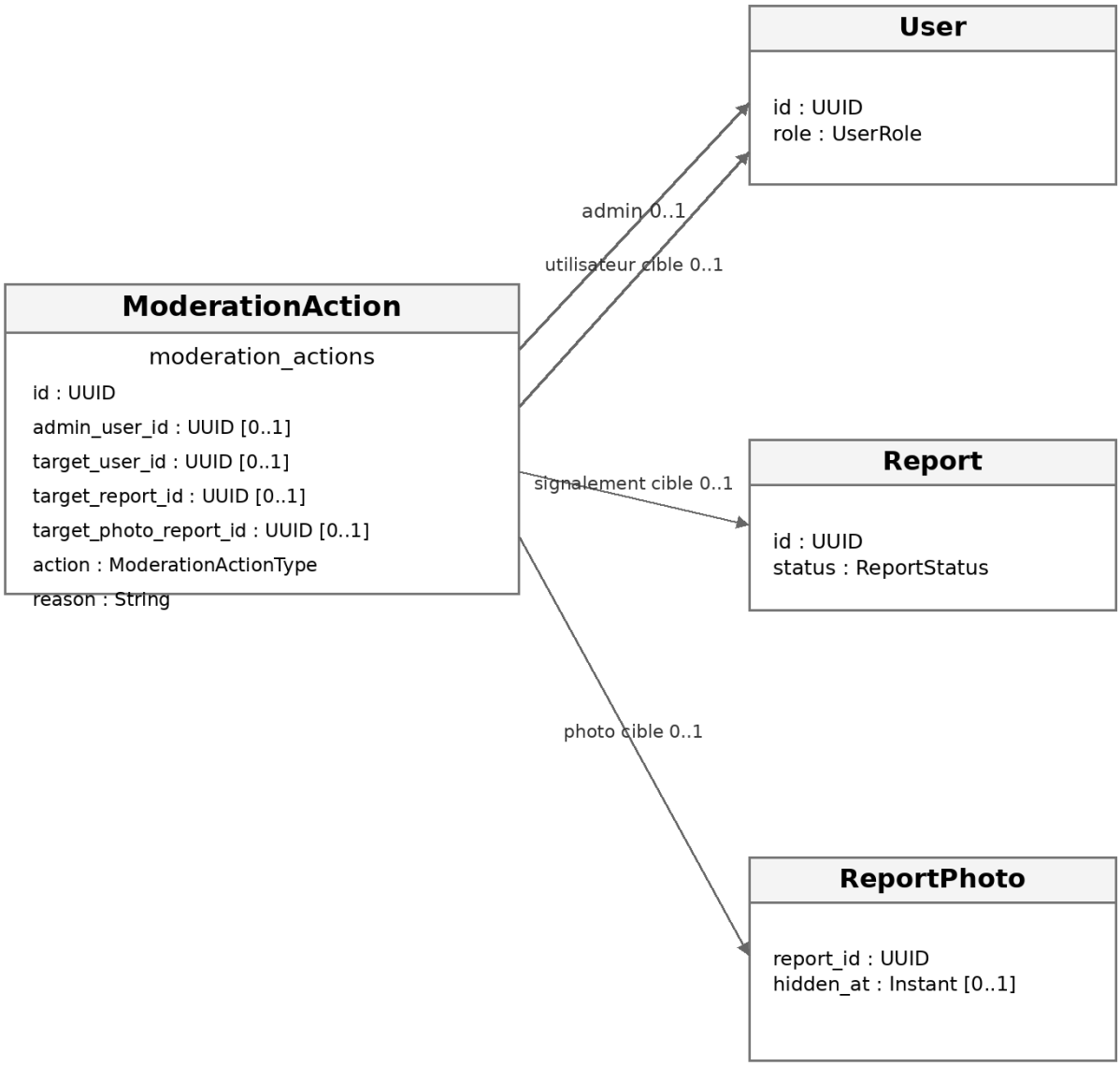


Planche A03 Diagramme complet

A04 Positionnement et création

Chapitre d'origine Séquence 1 publier un signalement photo Page 38

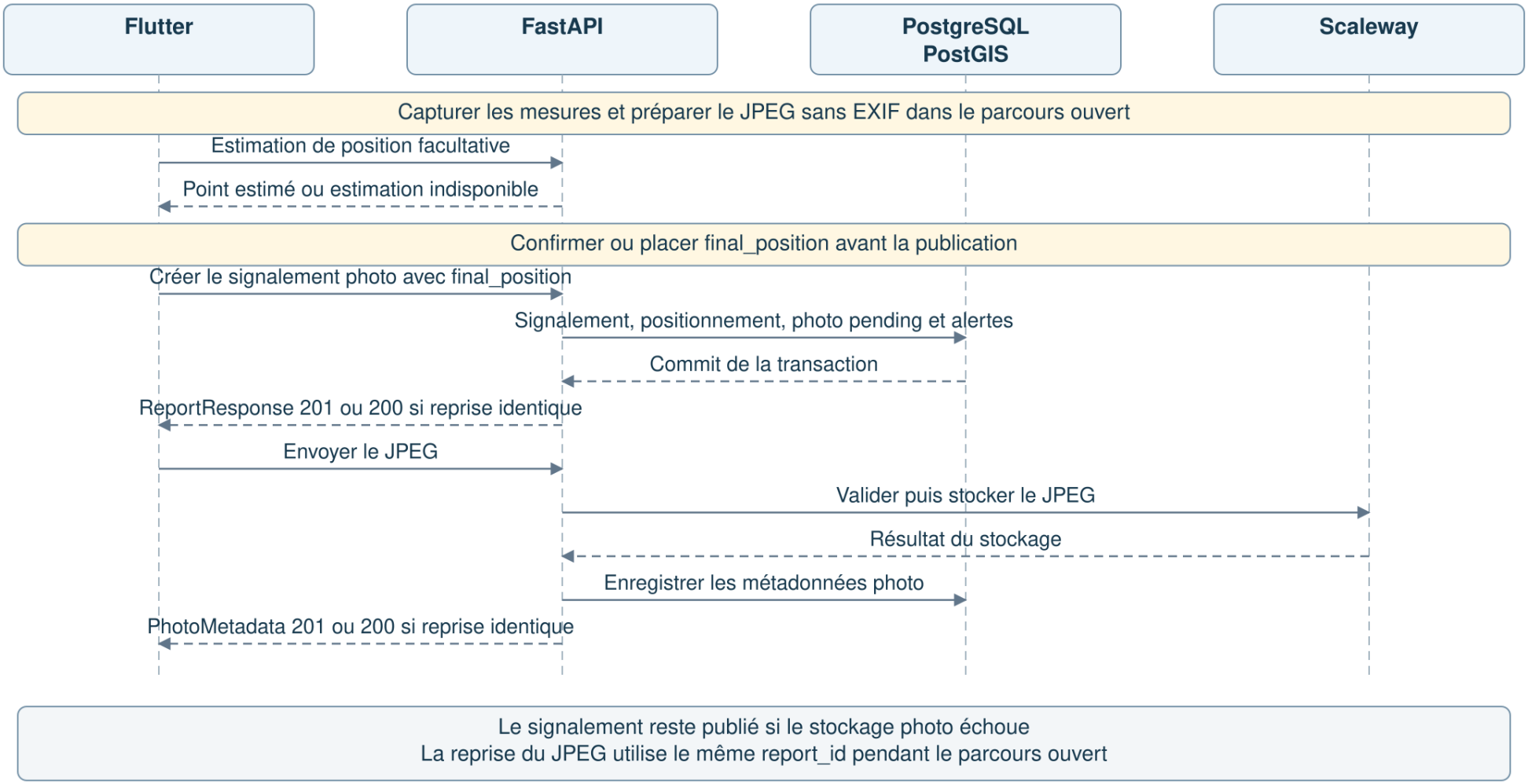


Planche A04 Diagramme complet

A05 Cartographie et consultation

Chapitre d'origine Séquence 2 consulter la carte Page 39

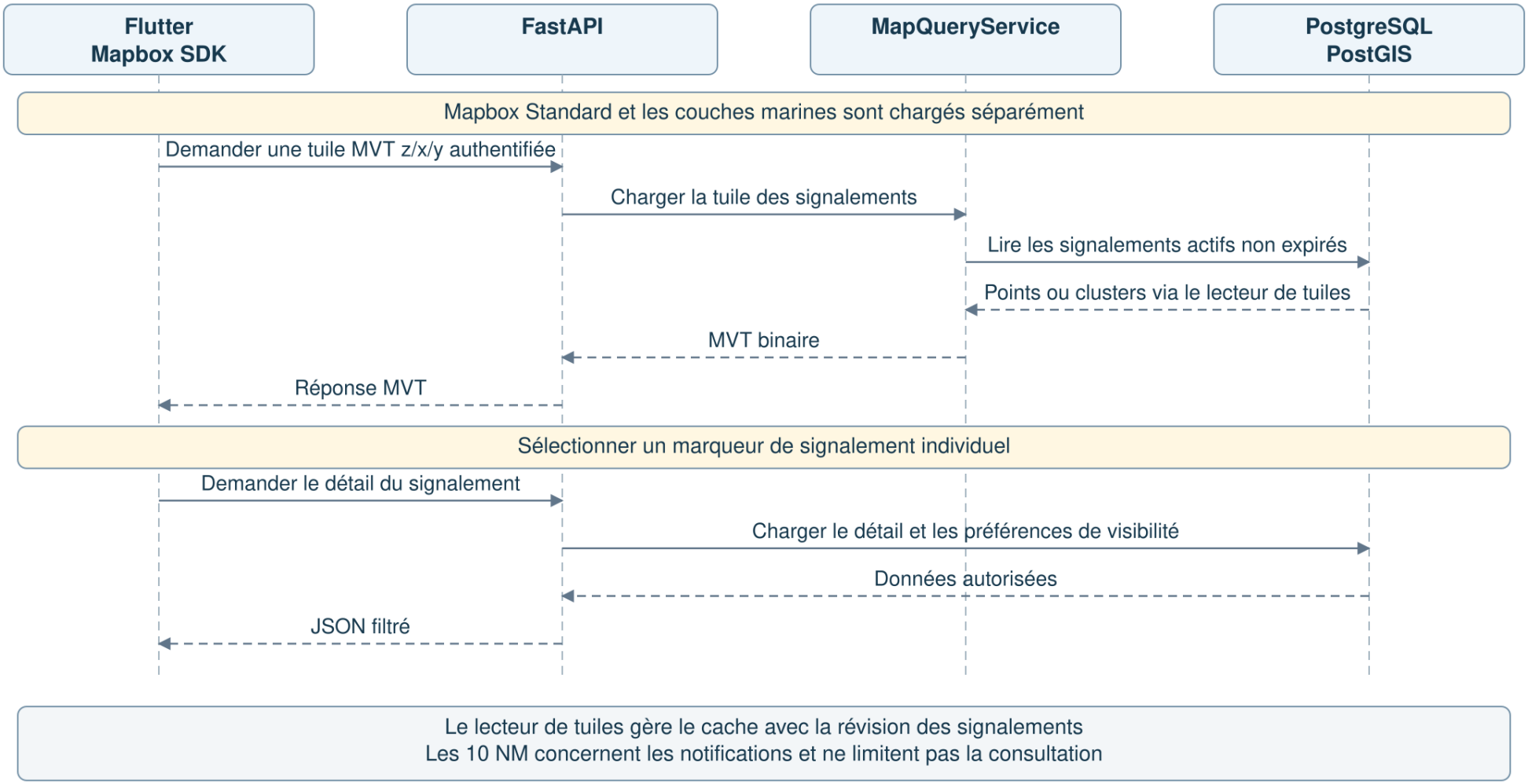


Planche A05 Diagramme complet

A06 Alertes et notifications

Chapitre d'origine Séquence 3 alerter les utilisateurs à proximité Page 40

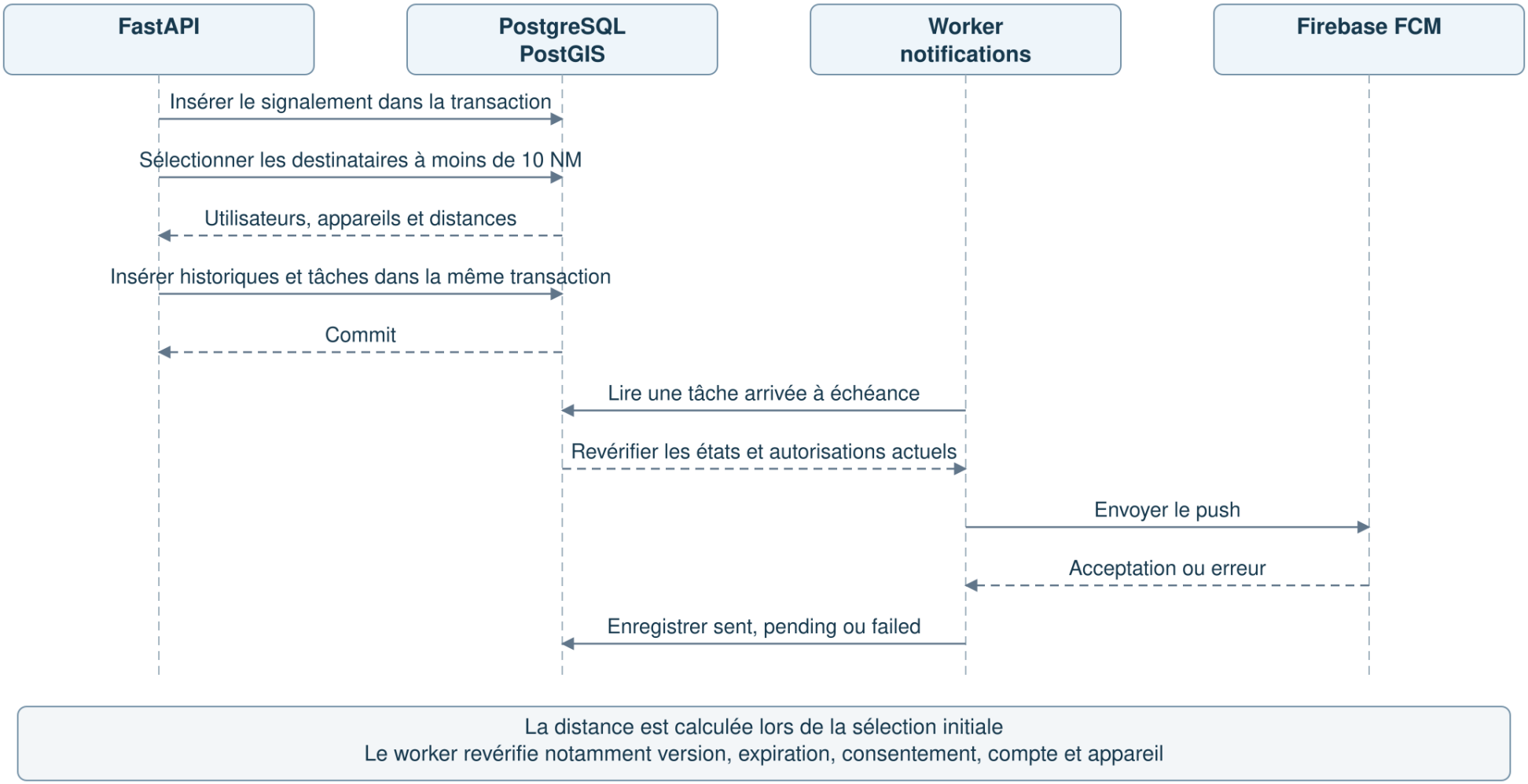


Planche A06 Diagramme complet

M01 MCD cœur métier

Chapitre d'origine Architecture du système puis MCD Page 12

MCD cœur métier

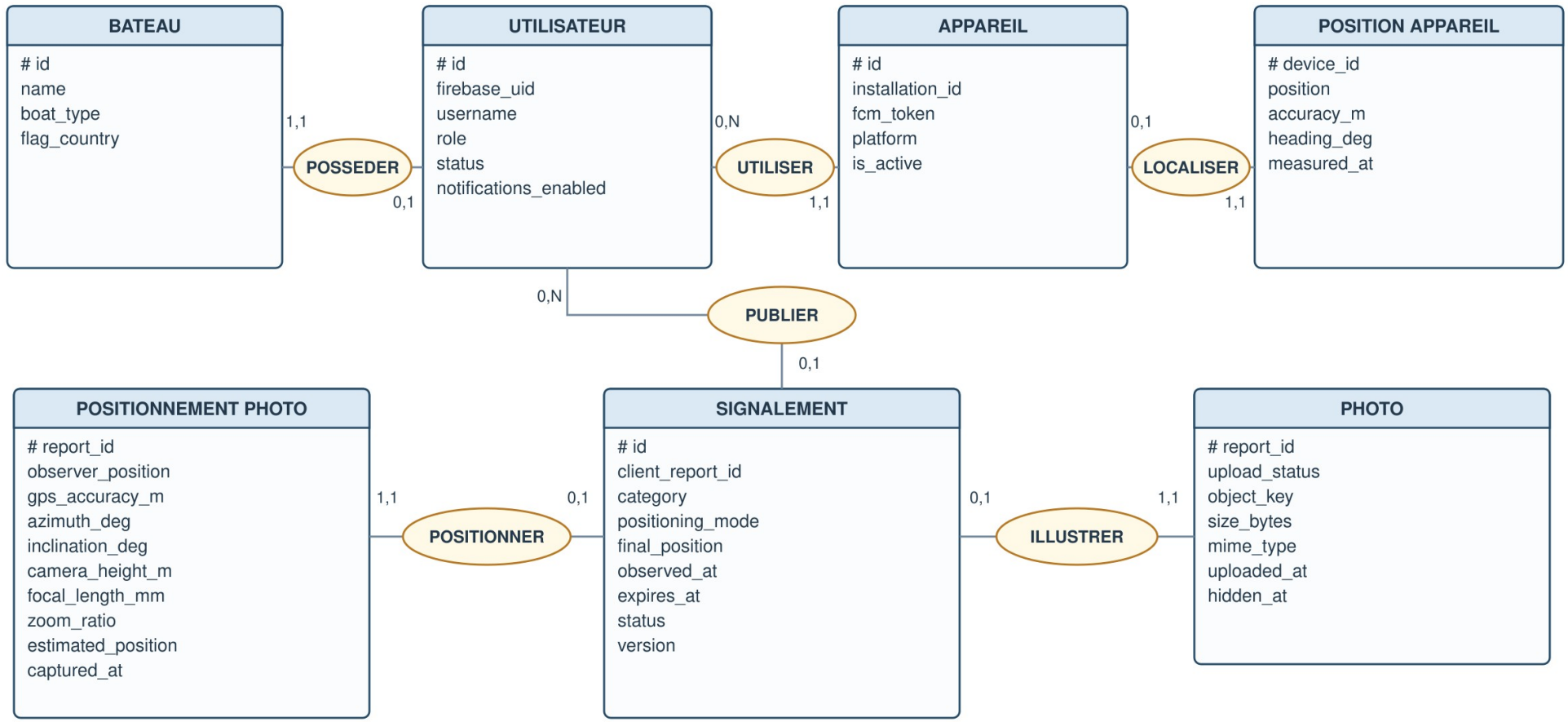


Planche M01 Partie 1 sur 3 du modèle conceptuel de données

0,1 zéro ou un 1,1 exactement un 0,N zéro ou plusieurs # identifiant conceptuel

M02 MCD notifications et historique

Chapitre d'origine Architecture du système puis MCD Page 12

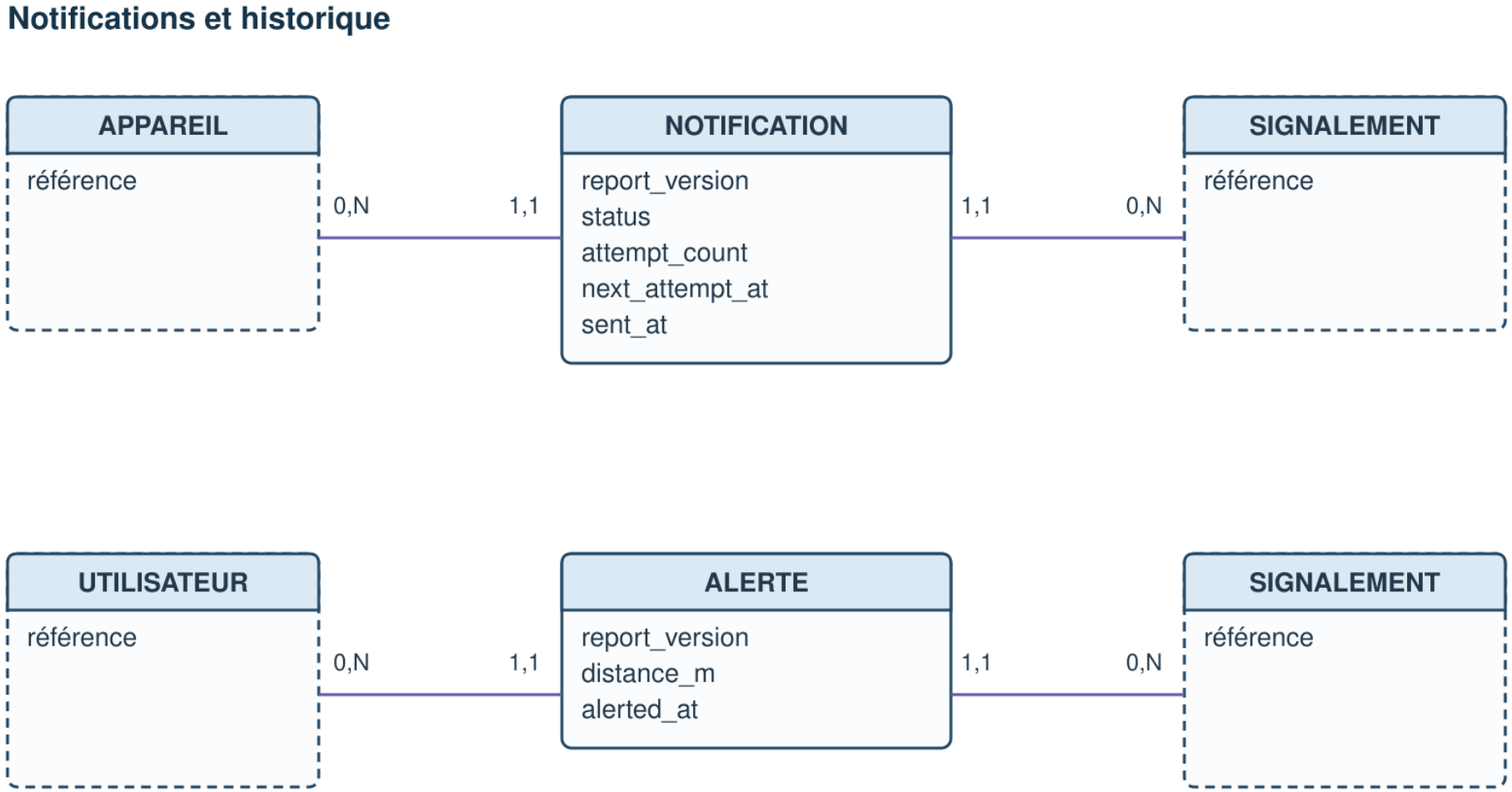


Planche M02 Partie 2 sur 3 du modèle conceptuel de données

0,1 zéro ou un 1,1 exactement un 0,N zéro ou plusieurs # identifiant conceptuel

Les boîtes pointillées désignent une référence à une entité déjà définie.

M03 MCD modération

Chapitre d'origine Architecture du système puis MCD Page 12

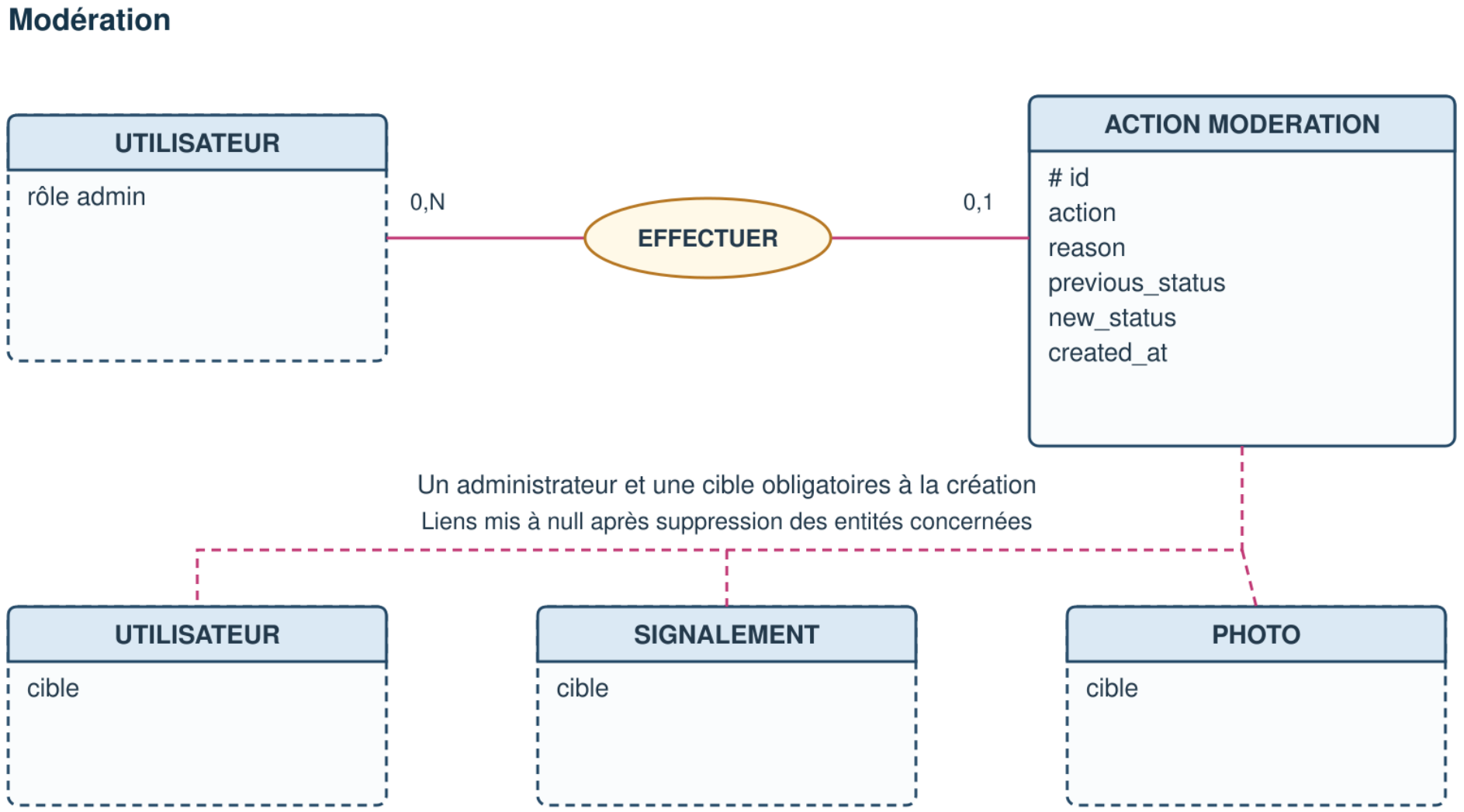


Planche M03 Partie 3 sur 3 du modèle conceptuel de données

0,1 zéro ou un 1,1 exactement un 0,N zéro ou plusieurs # identifiant conceptuel

Les boîtes pointillées désignent une référence à une entité déjà définie.